



UNIVERSITATEA DIN
BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ



SPECIALIZAREA MATEMATICĂ APLICATĂ

Lucrare de licență

METODA INDEXULUI PENTRU PROBLEMA LOGARITMULUI DISCRET

Absolvent

Georgescu Ștefania

Coordonator științific

Conf. dr. Dumitru Stamate

București, iunie 2025

Cuprins

Introducere	1
1 Preliminarii	3
1.1 Grupul ciclic $(\mathbb{F}_p^*, \cdot)$	3
1.2 Lema Chineză a Resturilor	5
1.3 Lema lui Hensel	6
2 Metoda indexului pentru problema logaritmului discret	10
2.1 Context	10
2.2 Prima etapă: aflarea logaritmilor elementelor din bază	12
2.3 A doua etapă: calcularea logaritmului discret	14
2.4 Exemplu	14
3 Complexitate	17
3.1 Notatii	17
3.2 Complexitatea metodei indexului	18
4 Implementare	25
4.1 Funcții ajutătoare	25
4.2 Funcția principală	29
4.2.1 Prima variantă de alcătuire a sistemului de congruențe	30
4.2.2 A doua variantă de alcătuire a sistemului de congruențe	31
4.3 Analiza comparativă a celor două variante de implementare	32
4.3.1 Interpretarea rezultatelor	33
4.3.2 Analiza boxplot	35
5 DLP în alte grupuri	38
5.1 Metoda indexului în $\mathbb{F}_{2^k}^*$	38
5.2 DLP pe curbe eliptice	40
Bibliografie	43

A	Codul pentru Varianta 1	45
B	Codul pentru Varianta 2	47

Introducere

Tema abordată în această lucrare este **metoda indexului**, prezentată atât din perspectivă teoretică, cât și practică, printr-o analiză comparativă a eficienței pentru două implementări proprii în Python. Metoda indexului este un algoritm utilizat pentru rezolvarea problemei logaritmului discret în grupul elementelor inversabile din corpul finit $\mathbb{F}_p$, cu p număr prim. Această problemă are importante aplicații practice în criptografie, întrucât dificultatea sa asigură, de exemplu, securitatea criptosistemului cu cheie publică ElGamal și a protocolului Diffie-Hellman. Importanța metodei indexului constă în faptul că evidențiază posibilele vulnerabilități ale acestor criptosisteme, fiind utilizată de criptografi în stabilirea unor parametri de securitate adecvați.

Am ales această temă deoarece mi-a plăcut cursul „Criptografie”, din semestrul al doilea al anului doi. Mi-a oferit o nouă perspectivă asupra matematicii și a utilității practice a acesteia, deschizându-mi noi posibilități pentru viitoarea mea carieră. Mi-a plăcut în mod special acest domeniu, deoarece combină două arii de interes, algebra și informatica.

În urma analizei teoretice a metodei indexului, mi-am asumat provocarea de a descoperi o variantă cât mai eficientă de implementare a acesteia și am dezvoltat două versiuni pe care le-am analizat și comparat în această lucrare. În plus, printre contribuțiile personale se numără și redactarea completă a demonstrației unui corolar (3.2.3) privind complexitatea metodei indexului, precum și adaptarea unor demonstrații ale unor teoreme fundamentale, pentru a le integra în contextul subiectului abordat. Referințele principale consultate pentru metoda indexului au fost [5] și [15]. Pentru aspectele matematice, am preluat, în plus, rezultate din [11].

Lucrarea este alcătuită din cinci capitole. Primul capitol surprinde câteva noțiuni și rezultate matematice fundamentale pentru înțelegerea metodei indexului, de exemplu Lema lui Hensel. În capitolul al doilea introducem problema logaritmului discret, care stă la baza acestei metode, și prezentăm metoda indexului în grupul finit $\mathbb{F}_p^*$. Sunt explicate etapele metodei și, în plus, am integrat un exemplu de utilizare. În capitolul următor prezentăm o analiză a complexității acestei metode. În capitolul al patrulea este detaliată implementarea proprie în Python. Totodată, pe baza a numeroase rulări, efectuăm o analiză comparativă a eficienței celor două variante implementate. Ultimul capitol surprinde, pe scurt, soluții la problema logaritmului discret în grupuri diferite de $\mathbb{F}_p^*$. La finalul

lucrării, am introdus în Bibliografie referințele consultate. Funcțiile principale din codul realizat în implementarea proprie sunt introduse în Anexele A și B. Codul complet este disponibil online, accesând [10].

Capitolul 1

Preliminarii

În acest capitol introducem noțiuni matematice fundamentale necesare înțelegerii metodei indexului, precum grupul ciclic $(\mathbb{F}_p^*, \cdot)$, reamintim Lema Chineză a Resturilor și prezentăm Lema lui Hensel, atât pe cazul general, cât și adaptată unui sistem de congruențe. Principalele referințe consultate au fost [15] și [11].

1.1 Grupul ciclic $(\mathbb{F}_p^*, \cdot)$

Următoarele rezultate din acest subcapitol sunt preluate și adaptate din [15].

Definiția 1.1.1. Fie $n \geq 2$ un număr natural. Notăm cu $\mathbb{Z}_n$ mulțimea claselor de resturi modulo n , iar $\mathbb{Z}_n^* = \mathbb{Z}_n \setminus \{\hat{0}\}$. Atunci $(\mathbb{Z}_n, +, \cdot)$ este un inel împreună cu operațiile de adunare și înmulțire modulo n . Elementul neutru la adunare este $\hat{0}$ și elementul neutru la înmulțire este $\hat{1}$.

Un număr $a \in \mathbb{Z}$ se numește **inversabil modulo n** dacă există un element $b \in \mathbb{Z}$ astfel încât $a \cdot b \equiv 1 \pmod{n}$. Notăm acest element cu a^{-1} .

Se știe că un element $a \in \mathbb{Z}$ este inversabil modulo n dacă și numai dacă $\text{cmmdc}(a, n) = 1$. Notăm cu $U(\mathbb{Z}_n) = \{\hat{x} \in \mathbb{Z}_n : \text{cmmdc}(x, n) = 1\}$ grupul elementelor inversabile din $\mathbb{Z}_n$.

Pentru p un număr prim, inelul $(\mathbb{Z}_p, +, \cdot)$ este un corp, deci $(\mathbb{Z}_p^*, \cdot)$ este un grup abelian finit de ordin $p - 1$.

Definiția 1.1.2 (Funcția φ a lui Euler). Pentru $n \geq 2$ un număr întreg, notăm prin $\varphi(n)$ numărul de întregi pozitivi a mai mici decât n cu proprietatea că $\text{cmmdc}(a, n) = 1$. În alte cuvinte, funcția $\varphi(n)$ numără câți întregi pozitivi mai mici decât n sunt coprimi cu n . Se consideră $\varphi(1) = 1$.

Dacă n are descompunerea în factori primi:

$$n = \prod_{i=1}^{\ell} p_i^{e_i},$$

unde p_i sunt numere prime distincte și $e_i \geq 1$, $i = 1, \dots, \ell$, atunci se poate demonstra:

$$\varphi(n) = n \prod_{i=1}^{\ell} \left(1 - \frac{1}{p_i}\right).$$

Avem următoarele cazuri particulare:

- (i) Dacă p este prim, atunci $\varphi(p) = p - 1$.
- (ii) Dacă p este prim și e un număr natural nenul, atunci $\varphi(p^e) = p^e \left(1 - \frac{1}{p}\right)$.
- (iii) Dacă $p_1, \dots, p_\ell$ sunt numere prime distincte, atunci

$$\varphi\left(\prod_{i=1}^{\ell} p_i\right) = \prod_{i=1}^{\ell} (p_i - 1).$$

Observația 1.1.3. Grupul multiplicativ $(U(\mathbb{Z}_n), \cdot)$ are ordinul $\varphi(n)$.

Teorema 1.1.4 (Teorema lui Euler). Fie $n \geq 2$ un număr natural. Atunci, oricare ar fi $a \in \mathbb{Z}$ cu $\text{cmmdc}(a, n) = 1$, avem:

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

În plus, avem consecința directă:

$$\hat{a}^{-1} = \hat{a}^{\varphi(n)-1} \text{ în } \mathbb{Z}_n.$$

Teorema lui Euler este o generalizarea a Miciei Teoreme a lui Fermat:

Teorema 1.1.5 (Mica Teoremă a lui Fermat). Fie p un număr prim și $a \in \mathbb{Z}$. Atunci avem:

$$a^p \equiv a \pmod{p}.$$

În plus, dacă $\text{cmmdc}(a, p) = 1$, avem:

$$a^{p-1} \equiv 1 \pmod{p}.$$

Definiția 1.1.6 (Grup ciclic). Un grup abelian $(X, \cdot)$ se numește *grup ciclic* dacă există un element $a \in X$ care generează întregul grup. Un astfel de element se numește *generator al grupului*.

Teorema 1.1.7. Fie $p \geq 2$ un număr prim. Atunci $(\mathbb{Z}_p^*, \cdot)$ este un grup ciclic de ordin $p - 1$, iar un generator al acestui grup se numește **element primitiv modulo p** .

Teorema 1.1.8. Fie $\hat{\alpha} \in \mathbb{Z}_p^*$. Atunci $\hat{\alpha}$ este un element primitiv modulo p dacă și numai dacă

$$\hat{\alpha}^{(p-1)/q} \neq 1$$

pentru toți divizorii primi q ai lui $p-1$.

Teorema 1.1.9. Orice corp finit este comutativ.

Teorema 1.1.10. Dacă un corp finit are q elemente, atunci $q = p^k$, unde p este un număr prim și $k \geq 1$ este un număr întreg. Reciproc, pentru orice q de forma p^k , cu p prim și $k \geq 1$, există un corp cu q elemente.

Teorema 1.1.11. Două corpuri finite cu același număr de elemente sunt izomorfe. Pentru orice q de forma p^k , unde p este un număr prim și $k \geq 1$ este un număr natural, vom nota unicul corp finit (până la un izomorfism) de ordin q cu $\mathbb{F}_q$.

Teorema 1.1.12. Grupul elementelor nenule dintr-un corp finit este ciclic.

Observația 1.1.13. Fie $g \in \mathbb{F}_p^*$ un element primitiv. Atunci:

$$\mathbb{F}_p^* = \{1, g, g^2, \dots, g^{p-2}\} \text{ și } (\mathbb{F}_p^*, \cdot) \simeq (\mathbb{Z}_{p-1}, +).$$

Observația 1.1.14. În această lucrare vom folosi simultan notațiile $\mathbb{F}_p^*$ și $\mathbb{Z}_p^*$ pentru mulțimea claselor nenule modulo p .

1.2 Lema Chineză a Resturilor

Noțiunile teoretice din acest subcapitol sunt prezentate conform [15].

Teorema 1.2.1. Fie $M = m_1 \cdot m_2 \cdot \dots \cdot m_r$, unde $m_1, \dots, m_r$ sunt numere naturale coprime două câte două, adică $\text{cmmdc}(m_i, m_j) = 1$ pentru toți $i \neq j$. Atunci inelul $(\mathbb{Z}_M, +, \cdot)$ este izomorf cu inelul $(\mathbb{Z}_{m_1} \times \dots \times \mathbb{Z}_{m_r}, +, \cdot)$.

Cu notațiile de mai sus, funcția $\chi : \mathbb{Z}_M \rightarrow \mathbb{Z}_{m_1} \times \dots \times \mathbb{Z}_{m_r}$ definită prin

$$\chi(a + M\mathbb{Z}) = (a \bmod m_1, \dots, a \bmod m_r),$$

este un izomorfism între inelele $(\mathbb{Z}_M, +, \cdot)$ și $(\mathbb{Z}_{m_1} \times \dots \times \mathbb{Z}_{m_r}, +, \cdot)$.

Observația 1.2.2. Pentru $1 \leq i \leq r$, definim $M_i = M/m_i$ și $y_i \equiv M_i^{-1} \bmod m_i$. Atunci funcția inversă $\chi^{-1} : \mathbb{Z}_{m_1} \times \dots \times \mathbb{Z}_{m_r} \rightarrow \mathbb{Z}_M$ este:

$$\chi^{-1}(\hat{a}_1, \dots, \hat{a}_r) = \widehat{\sum_{i=1}^r a_i M_i y_i}.$$

Teorema 1.2.3 (Lema Chineză a Resturilor). Fie $m_1, \dots, m_r$ numere întregi pozitive coprime două câte două, iar $a_1, \dots, a_r$ sunt numere întregi. Atunci sistemul de r congruențe:

$$x \equiv a_i \pmod{m_i} \quad i = 1, \dots, r$$

are o soluție unică modulo $M = m_1 \cdot m_2 \cdot \dots \cdot m_r$, și anume $\hat{x} = \chi^{-1}(\hat{a}_1, \dots, \hat{a}_r)$.

Exemplul 1.2.4. Pentru rezolvarea sistemului de congruențe:

$$x \equiv 3 \pmod{16}$$

$$x \equiv 2 \pmod{15}$$

$$x \equiv 4 \pmod{7},$$

avem $m_1 = 16$, $m_2 = 15$, $m_3 = 7$, iar $M = m_1 \cdot m_2 \cdot m_3 = 1680$. Calculăm $M_1 = M/m_1 = 105$, $M_2 = M/m_2 = 112$, $M_3 = M/m_3 = 240$, deci, cu notațiile de mai sus:

$$y_1 \equiv M_1^{-1} \equiv 9 \pmod{m_1}$$

$$y_2 \equiv M_2^{-1} \equiv 13 \pmod{m_2}$$

$$y_3 \equiv M_3^{-1} \equiv 4 \pmod{m_3}.$$

Soluția sistemului de congruențe satisface:

$$x \equiv 3 \cdot 105 \cdot 9 + 2 \cdot 112 \cdot 13 + 4 \cdot 240 \cdot 4 \equiv 2835 + 2912 + 3840 \equiv 1187 \pmod{1680}.$$

Echivalent, mulțimea soluțiilor întregi este:

$$S = \{1187 + 1680k : k \in \mathbb{Z}\}.$$

1.3 Lema lui Hensel

În implementarea metodei indexului, întâlnim situația în care este necesar să aflăm soluția unui sistem de congruențe modulo p^k , pentru p un număr prim și $k \geq 2$ natural. Abordarea acestei probleme constă în rezolvarea sistemului modulo p , urmată apoi de “ridicarea soluției” în mod succesiv modulo $p^2, p^3, \dots, p^k$.

Vom studia mai întâi cazul în care avem de rezolvat o singură congruență. Faptul că este posibil să “ridicăm” o soluție modulo p^k este garantat de Lema lui Hensel. În acest sens, avem următorul rezultat, preluat și adaptat din [11, pp. 104]:

Teorema 1.3.1 (Lema lui Hensel). Fie p un număr prim, $n \in \mathbb{N}^*$ și $f \in \mathbb{Z}[X]$ un polinom astfel încât există $a_n \in \mathbb{Z}$ cu $f(a_n) \equiv 0 \pmod{p^n}$ și $f'(a_n) \not\equiv 0 \pmod{p}$. Atunci

există $a_{n+1} \in \mathbb{Z}$, unic modulo p^{n+1} , astfel încât $f(a_{n+1}) \equiv 0 \pmod{p^{n+1}}$ și $a_{n+1} \equiv a_n \pmod{p^n}$.

Demonstrație. Știm că $a_{n+1} \equiv a_n \pmod{p^n}$, deci există $t_n \in \mathbb{Z}$, astfel încât $a_{n+1} = a_n + t_n p^n$. Căutăm t_n unic modulo p , astfel încât $f(a_{n+1}) \equiv 0 \pmod{p^{n+1}}$.

Pentru a calcula $f(a_{n+1}) = f(a_n + t_n p^n)$ ne folosim de dezvoltarea în serie Taylor a polinomului f cu $x = a_n + t_n p^n$ și $x_0 = a_n$:

$$f(x) = f(x_0) + \frac{f'(x_0)}{1!}(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \frac{f^{(3)}(x_0)}{3!}(x - x_0)^3 + \dots$$

$$f(a_n + t_n p^n) = f(a_n) + \frac{f'(a_n)}{1!}t_n p^n + \frac{f''(a_n)}{2!}(t_n p^n)^2 + \dots$$

Este ușor de observat că $\frac{f^{(r)}(a_n)}{r!} \in \mathbb{Z}$, pentru oricare $r \in \mathbb{N}^*$ astfel încât $1 \leq r \leq \deg f$. În plus, $2n \geq n + 1$, pentru orice $n \geq 1$, așadar, obținem:

$$f(a_n + t_n p^n) \equiv f(a_n) + f'(a_n)t_n p^n \pmod{p^{n+1}}.$$

Cu această scriere avem:

$$\begin{aligned} f(a_{n+1}) = f(a_n + t_n p^n) &\equiv 0 \pmod{p^{n+1}} \\ \Leftrightarrow f(a_n) + f'(a_n)t_n p^n &\equiv 0 \pmod{p^{n+1}} \\ \Leftrightarrow f'(a_n)t_n p^n &\equiv -f(a_n) \pmod{p^{n+1}}. \end{aligned}$$

Dar $f(a_n) \equiv 0 \pmod{p^n}$, deci există $k \in \mathbb{Z}$ astfel încât $f(a_n) = kp^n$ și $k = \frac{f(a_n)}{p^n} \in \mathbb{Z}$. Astfel, avem:

$$\begin{aligned} f'(a_n)t_n p^n &\equiv -p^n \cdot \frac{f(a_n)}{p^n} \pmod{p^{n+1}} \text{ și} \\ f'(a_n)t_n &\equiv -\frac{f(a_n)}{p^n} \pmod{p}. \end{aligned} \tag{1.1}$$

Cum $f'(a_n) \not\equiv 0 \pmod{p}$, rezultă că $\widehat{f'(a_n)}$ este inversabil în $\mathbb{Z}_p$ și notăm $f'(a_n)^{-1}$ numărul întreg, unic modulo p , pentru care $\widehat{f'(a_n)}\widehat{f'(a_n)^{-1}} = \widehat{1}$ în $\mathbb{Z}_p$. Ecuația (1.1) devine:

$$t_n \equiv -(f'(a_n))^{-1} \frac{f(a_n)}{p^n} \pmod{p}. \tag{1.2}$$

Așadar, t_n este unic modulo p și obținem:

$$a_{n+1} \equiv a_n - (f'(a_n))^{-1} \frac{f(a_n)}{p^n} p^n \pmod{p^{n+1}}, \tag{1.3}$$

unde a_{n+1} este unic modulo p^{n+1} .

□

Rezolvarea de sisteme de congruențe

Putem extinde Lema lui Hensel la nivelul unui sistem de congruențe și obținem următoarea problemă:

Problema 1.3.2. Fie p un număr prim, $n \in \mathbb{N}^*$ și fie sistemul de m congruențe modulo p^n dat de:

$$A\mathbf{x}_n \equiv \mathbf{b} \pmod{p^n}, \quad (1.4)$$

unde matricea $A \in \mathcal{M}_m(\mathbb{Z})$, $\det A \not\equiv 0 \pmod{p}$, iar $\mathbf{b} \in \mathcal{M}_{m \times 1}(\mathbb{Z})$ este coloana termenilor liberi. Știind că există vectorul $\mathbf{x}_n \in \mathcal{M}_{m \times 1}(\mathbb{Z})$, soluție modulo p^n a sistemului de congruențe (1.4), vrem să putem extinde această soluție modulo p^{n+1} . În alte cuvinte, căutăm $\mathbf{x}_{n+1} \in \mathcal{M}_{m \times 1}(\mathbb{Z})$, cu $\mathbf{x}_n \equiv \mathbf{x}_{n+1} \pmod{p^n}$, astfel încât:

$$A\mathbf{x}_{n+1} \equiv \mathbf{b} \pmod{p^{n+1}}, \quad (1.5)$$

iar $\mathbf{x}_{n+1}$ să fie determinat în mod unic modulo p^{n+1} .

Vrem să găsim o formulă de calcul a lui $\mathbf{x}_{n+1}$, precum am găsit pentru a_{n+1} expresia (1.3) în demonstrația Teoremei 1.3.1. Așadar, căutăm $\mathbf{t}_n \in \mathcal{M}_{m \times 1}(\mathbb{Z})$ unic modulo p , pentru care:

$$\mathbf{x}_{n+1} = \mathbf{x}_n + p^n \mathbf{t}_n.$$

Vrem $A\mathbf{x}_{n+1} \equiv \mathbf{b} \pmod{p^{n+1}}$, deci:

$$\begin{aligned} A(\mathbf{x}_n + p^n \mathbf{t}_n) &\equiv \mathbf{b} \pmod{p^{n+1}} \\ \Leftrightarrow A\mathbf{x}_n + p^n A\mathbf{t}_n &\equiv \mathbf{b} \pmod{p^{n+1}} \\ \Leftrightarrow p^n A\mathbf{t}_n &\equiv \mathbf{b} - A\mathbf{x}_n \pmod{p^{n+1}}. \end{aligned}$$

Știm că $A\mathbf{x}_n \equiv \mathbf{b} \pmod{p^n}$, deci $\mathbf{b} - A\mathbf{x}_n \equiv \mathbf{0} \pmod{p^n}$. Așadar, există $\mathbf{w}_n \in \mathcal{M}_{m \times 1}(\mathbb{Z})$ astfel încât $\mathbf{b} - A\mathbf{x}_n = p^n \mathbf{w}_n$ și $\mathbf{w}_n = \frac{1}{p^n}(\mathbf{b} - A\mathbf{x}_n)$.

Cu această notație, putem scrie: $p^n A\mathbf{t}_n \equiv p^n \mathbf{w}_n \pmod{p^{n+1}}$, echivalent cu $A\mathbf{t}_n \equiv \mathbf{w}_n \pmod{p}$.

Deoarece matricea A are determinantul inversabil modulo p , atunci există și este unică modulo p matricea $A^{-1} \in \mathcal{M}_m(\mathbb{Z})$ astfel încât $AA^{-1} \equiv A^{-1}A \equiv I_m \pmod{p}$. Astfel, obținem:

$$\mathbf{t}_n \equiv A^{-1}\mathbf{w}_n \pmod{p} \quad (1.6)$$

unic modulo p , iar soluția sistemului de congruențe (1.5) este

$$\mathbf{x}_{n+1} \equiv \mathbf{x}_n + p^n A^{-1}\mathbf{w}_n \pmod{p^{n+1}}, \quad (1.7)$$

unică modulo p^{n+1} .

Mai multe detalii privind folosirea Lemei lui Hensel la rezolvarea de sisteme de congruențe pot fi găsite în articolul [4].

Capitolul 2

Metoda indexului pentru problema logaritmului discret

În acest capitol introducem problema logaritmului discret într-un grup oarecare. Prezentăm metoda indexului care rezolvă problema logaritmului discret într-un grup de tip $\mathbb{F}_p^*$, cu p prim. În continuare, descriem structura și etapele principale ale metodei, însoțite de un exemplu de utilizare. Pentru elaborarea acestui capitol, am consultat și adaptat informații din lucrarea [5].

2.1 Context

În contextul numerelor reale, logaritmul este operația inversă exponențierii. Date $x \in \mathbb{R}^+ \setminus \{0\}$ și $b \in \mathbb{R}^+ \setminus \{0, 1\}$, funcția logaritm (real) în baza b este definită astfel:

$$\log_b(\cdot) : \mathbb{R}^+ \setminus \{0\} \rightarrow \mathbb{R}, \log_b(x) = y \iff b^x = y.$$

Dacă privim logaritmul în contexte diferite, precum grupurile finite, determinarea soluției ecuației devine un proces mai complex. Totuși, această complexitate oferă oportunități pentru formularea unor probleme matematice dificile (în sensul duratei de rezolvare), având aplicații semnificative în criptografie.

Definiția 2.1.1. Dat $(G, \cdot)$ un grup finit și $g \in G, g \neq 1$, pentru $h \in G$, numim logaritmul său discret, dacă există, un element $x \in \mathbb{Z}$ pentru care:

$$g^x = h. \tag{2.1}$$

Observația 2.1.2. Ecuația (2.1) are soluție dacă și numai dacă h aparține subgrupului generat de g .

Observația 2.1.3. Fie $(G, \cdot)$ un grup, $g, h \in G$, cu $N = \text{ord}(g)$ ordinul elementului g . Dacă $N < \infty$ și x_0 este soluție a ecuației $g^x = h$, atunci orice altă soluție este de forma

$x_0 + kN$, cu $k \in \mathbb{Z}$ arbitrar. Așadar, soluția pentru $g^x = h$ este unic determinată modulo $N = \text{ord}(g)$.

Demonstrație. $g^{x_0+kN} = g^{x_0} \cdot g^{kN} = g^{x_0} \cdot (g^N)^k = g^{x_0} \cdot 1 = g^{x_0} = h$, deoarece $N = \text{ord}(g)$ și x_0 este soluție a ecuației $g^x = h$ în $(G, \cdot)$. $\square$

Exemplul 2.1.4. Fie p un număr prim și grupul $G = (\mathbb{Z}_p, +)$. Fixăm $\hat{g} \neq \hat{0}$, $\hat{g} \in \mathbb{Z}_p$ și $\hat{h} \in \mathbb{Z}_p$. Problema logaritmului discret constă în aflarea lui $x \in \mathbb{Z}$ pentru care

$$x \cdot \hat{g} = \hat{h}.$$

Rezolvarea, în acest caz, este ușoară. Cum p este prim, $(\mathbb{Z}_p, +, \cdot)$ este corp, iar elementul $\hat{g}$ este inversabil, așadar există $\hat{g}^{-1} \in \mathbb{Z}_p$ și $\hat{x} = \hat{g}^{-1} \cdot \hat{h}$.

În această lucrare ne vom concentra pe rezolvarea problemei logaritmului discret în grupul $(G, \cdot) = (\mathbb{F}_p^*, \cdot)$, unde p este un număr prim. Formulăm următoarea definiție:

Definiția 2.1.5. Fie p un număr prim, $g \in \mathbb{Z}$ o rădăcină primitivă în $\mathbb{F}_p^*$ și fie h un număr întreg astfel încât $h \not\equiv 0 \pmod{p}$. **Problema logaritmului discret (DLP)** este problema determinării unui exponent $x \in \mathbb{Z}$ astfel încât:

$$g^x \equiv h \pmod{p}. \quad (2.2)$$

Numărul x se numește **logaritm discret** al lui h în baza g și se notează:

$$\log_g h.$$

O altă denumire întâlnită în teoria numerelor pentru logaritmul discret este aceea de **indice** sau **index**.

Observația 2.1.6. Cu notațiile de mai sus, cum $\text{ord}(g) = p - 1$, dacă avem $x_0 \in \mathbb{Z}$ o soluție particulară a ecuației (2.2), ea ne furnizează mulțimea de soluții: $S = \{x_0 + k(p - 1) : k \in \mathbb{Z}\}$, deoarece:

$$g^{x_0+k(p-1)} = g^{x_0} \cdot (g^{p-1})^k \equiv h \cdot 1^k \equiv h \pmod{p}.$$

Așadar, putem spune că problema logaritmului discret are soluții congruente modulo $p - 1$, iar, mai departe, vom considera “soluția problemei logaritmului discret” unica soluție $x_0 = \log_g h$ din intervalul $[0, p - 2]$.

Observația 2.1.7. $\log_g(ab) \equiv \log_g a + \log_g b \pmod{p - 1}$, pentru orice $a, b \in \mathbb{Z}$, cu a și b diferite de 0 modulo p .

Metoda cea mai evidentă de rezolvare a problemei logaritmului discret este căutarea exhaustivă. Dată ecuația $g^x \equiv h \pmod{p}$, unde $g, h \in \mathbb{Z}$, g rădăcină primitivă în grupul $\mathbb{Z}_p^*$, calculăm $g^1, g^2, g^3, \dots$ până găsim exponentul i pentru care $g^i \equiv h \pmod{p}$. Această metodă, deși sigură, își pierde însă eficiența atunci când lucrăm cu numere mari. Există metode mai eficiente de rezolvare a DLP, cum ar fi: algoritmul *Baby-step, giant-step* introdus de Shanks în 1971 [14], algoritmul Pohlig-Hellman [12] sau metoda Pollard Rho din 1978 [13]. Algoritmul pe care se va concentra această lucrare este Metoda indexului.

Metoda indexului este un algoritm conceput pentru rezolvarea problemei logaritmului discret (2.2) în grupul finit $\mathbb{F}_p^*$. Prima versiune a fost dezvoltată în 1979 de Leonard M. Adleman [1], care a propus o abordare subexponențială. Ulterior, în 1985, Maurer și Odlyzko [8] au îmbunătățit eficiența metodei, iar în 1991, LaMacchia și Odlyzko [7] au propus noi optimizări.

Algoritmul pentru metoda indexului necesită fixarea unei baze alcătuite din numere prime mici. Alegem P un număr prim și $\mathcal{B}$ mulțimea alcătuită din toate numerele prime mai mici sau egale cu P :

$$\mathcal{B} = \{p_1, p_2, \dots, p_B = P\}, \text{ deci } |\mathcal{B}| = B.$$

Unul dintre aspectele fundamentale ale metodei indexului constă în identificarea unor numere cu o factorizare particulară. Aceste numere au o denumire specifică:

Definiția 2.1.8 (Număr $\mathcal{B}$ -smooth). Fie baza $\mathcal{B}$ definită mai sus. Un număr natural n se numește $\mathcal{B}$ -smooth dacă toți factorii primi din descompunerea aparțin mulțimii $\mathcal{B}$.

Metoda indexului are două etape principale. Prima etapă urmărește aflarea tuturor logaritmilor de forma $\log_g p_i$, $i \in \{1, 2, \dots, B\}$, ce vor fi folosiți, ulterior, în a doua etapă, pentru calcularea soluției $x_0 = \log_g h$ a problemei logaritmului discret 2.2.

2.2 Prima etapă: aflarea logaritmilor elementelor din bază

În prima etapă a metodei indexului vrem să aflăm logaritmii numerelor prime din baza $\mathcal{B}$, adică $\log_g p_i$, pentru $i \in \{1, 2, \dots, B\}$.

Fie C un număr natural, $C \geq B$. Căutăm C numere întregi de forma:

$$g_i \equiv g^{t_i} \pmod{p}, \quad 0 \leq g_i \leq p-1$$

pentru niște exponenți t_i aleși în mod aleator în intervalul $[1, p-2]$, astfel încât aceste numere g_i să fie $\mathcal{B}$ -smooth. Astfel, ele pot fi scrise în funcție de elementele din baza $\mathcal{B}$

ca:

$$g_i \equiv g^{t_i} \equiv p_1^{a_{1i}} \cdot p_2^{a_{2i}} \cdot \dots \cdot p_B^{a_{Bi}} \pmod{p}, \quad i \in 1, \dots, C, \quad a_{1i}, a_{2i}, \dots, a_{Bi} \in \mathbb{N}. \quad (2.3)$$

Echivalent, putem scrie:

$$t_i \equiv \log_g g_i \equiv a_{1i} \log_g p_1 + a_{2i} \log_g p_2 + \dots + a_{Bi} \log_g p_B \pmod{p-1}, \quad i \in 1, \dots, C. \quad (2.4)$$

Notând $\log_g p_j = \ell_j$, $j \in 1, \dots, B$, obținem un sistem cu C congruențe modulo $p-1$ și B necunoscute:

$$t_i \equiv a_{1i} \ell_1 + a_{2i} \ell_2 + \dots + a_{Bi} \ell_B \pmod{p-1}, \quad i \in 1, \dots, C. \quad (S_1)$$

Pentru a putea obține soluție unică modulo $p-1$, matricea asociată sistemului de congruențe (S_1) trebuie să aibă un minor de rang maxim inversabil modulo $p-1$, iar, din felul în care a fost construit sistemul, avem garantată compatibilitatea. Astfel, selectăm din cele C congruențe, B congruențe care să alcătuiască un nou sistem pătratic compatibil determinat. Soluțiile acestuia vor fi logaritmi căutați $\log_g p_j = \ell_j$, $j \in 1, \dots, B$. Renotând indicii din congruențele selectate cu $k \in 1, \dots, B$, noul sistem obținut este:

$$t_k \equiv a_{1k} \ell_1 + a_{2k} \ell_2 + \dots + a_{Bk} \ell_B \pmod{p-1}, \quad k \in 1, \dots, B. \quad (S_2)$$

Matricea asociată sistemului, $A = (a_{mn})_{m,n \in \overline{1,B}}$, este pătratică și are determinantul inversabil în $\mathbb{Z}_{p-1}$, fiind formată din congruențele care alcătuiau minorul principal ales.

În cazul în care matricea asociată sistemului (S_1) nu are rang maxim, trebuie să alegem un număr $C' > C$, să generăm $C' - C$ numere $\mathcal{B}$ -smooth suplimentare pentru a completa sistemul (S_1) cu noile congruențe obținute, până când vom obține matricea asociată sistemului (S_1) să fie de rang maxim. Ulterior, vom continua cu generarea sistemului (S_2) ca mai sus.

În ceea ce privește rezolvarea sistemului de congruențe, metoda de eliminare Gauss poate fi deficitară, deoarece lucrăm modulo $p-1$, care este un număr compus și nu toate numerele sunt inversabile modulo $p-1$. De aceea, vom descompune $p-1$ în factori primi și vom rezolva sistemul modulo fiecare factor prim în parte. La final, combinăm soluțiile modulo factorii primi pentru a obține soluția modulo $p-1$, folosindu-ne de Lema Chineză a Resturilor 1.2.3.

În cazul în care apare q^e în descompunerea lui $p-1$, unde q este prim și $e > 1$, atunci ne folosim de Lema lui Hensel 1.3.1 pentru a ridica soluția din $\mathbb{Z}_q$ în $\mathbb{Z}_{q^e}$. Odată aflați $\log_g p_j = \ell_j$, $j \in 1, \dots, B$, putem trece la a doua etapă a metodei indexului, ce presupune aflarea lui $\log_g h$.

2.3 A doua etapă: calcularea logaritmului discret

Pentru a afla soluția problemei logaritmului discret (2.2), mai întâi trebuie să găsim un nou număr $\mathcal{B}$ – *smooth* de forma $h \cdot g^s \pmod{p}$, pentru un exponent s ales în mod aleator în intervalul $[1, p-2]$. Pentru acest s ales convenabil, putem scrie:

$$h \cdot g^s \equiv \prod_{i=1}^B p_i^{e_i} \pmod{p}$$

pentru anumiți exponenți e_i .

Logaritmând această expresie, obținem:

$$\begin{aligned} \log_g(h \cdot g^s) &\equiv \log_g\left(\prod_{i=1}^B p_i^{e_i}\right) \pmod{p-1} \\ \Leftrightarrow (\log_g h) + s &\equiv \sum_{i=1}^B \log_g p_i^{e_i} \pmod{p-1} \\ \Leftrightarrow \log_g h &\equiv -s + \sum_{i=1}^B e_i \log_g p_i \pmod{p-1}. \end{aligned}$$

Cum am calculat deja toți $\log_g p_i$, $i \in 1, \dots, B$, am aflat $x_0 = \log_g h$, unica soluție a problemei logaritmului discret $g^x \equiv h \pmod{p}$ din intervalul $[0, p-2]$, iar mulțimea tuturor soluțiilor este:

$$S = \{x_0 + k(p-1) : k \in \mathbb{Z}\}.$$

2.4 Exemplu

Fie numărul prim $p = 22613$ și fixăm baza $\mathcal{B} = \{2, 3, 5, 7\}$. În plus, fie $h = 578$ și $g = 17$ rădăcină primitivă în $\mathbb{F}_{22613}^*$. Vrem să folosim metoda indexului pentru a rezolva problema logaritmului discret:

$$17^x \equiv 578 \pmod{22613}. \quad (2.5)$$

Începem prin a calcula $g^t \pmod{p}$, unde exponentul t este ales în mod aleator, pentru a căuta numere $\mathcal{B}$ – *smooth*. După mai multe încercări, obținem următoarele numere $\mathcal{B}$ – *smooth*:

$$\begin{aligned} 4116 &\equiv g^{2588} \equiv 2^2 \cdot 3 \cdot 7^3 \pmod{22613}, \\ 700 &\equiv g^{12213} \equiv 2^2 \cdot 5^2 \cdot 7 \pmod{22613}, \\ 7290 &\equiv g^{18154} \equiv 2 \cdot 3^6 \cdot 5 \pmod{22613}, \\ 6860 &\equiv g^{12836} \equiv 2^2 \cdot 5 \cdot 7^3 \pmod{22613}. \end{aligned}$$

Logaritmând, obținem sistemul de congruențe:

$$\begin{aligned}
2588 &\equiv 2 \log_g 2 + \log_g 3 + 3 \log_g 7 \pmod{22612}, \\
12213 &\equiv 2 \log_g 2 + 2 \log_g 5 + \log_g 7 \pmod{22612}, \\
18154 &\equiv \log_g 2 + 6 \log_g 3 + \log_g 5 \pmod{22612}, \\
12836 &\equiv 2 \log_g 2 + \log_g 5 + 3 \log_g 7 \pmod{22612}.
\end{aligned} \tag{S_1}$$

Notăm $x_2 = \log_g 2$, $x_3 = \log_g 3$, $x_5 = \log_g 5$ și $x_7 = \log_g 7$. Sistemul (S_1) devine:

$$\begin{aligned}
2588 &\equiv 2x_2 + x_3 + 3x_7 \pmod{22612}, \\
12213 &\equiv 2x_2 + 2x_5 + x_7 \pmod{22612}, \\
18154 &\equiv x_2 + 6x_3 + x_5 \pmod{22612}, \\
12836 &\equiv 2x_2 + x_5 + 3x_7 \pmod{22612}.
\end{aligned} \tag{S_2}$$

Cum determinantul matricii sistemului este $\det A = -23$, iar $-23 \equiv 22589 \pmod{22612}$ este inversabil modulo 22612, (S_2) este compatibil determinat și are soluție unică modulo 22612. În plus, sistemul de congruențe (S_2) este modulo $p - 1 = 22612$, care se descompune în $22612 = 2^2 \cdot 5653$, unde 5653 este prim. Așadar, rezolvăm prin metoda lui Gauss sistemul modulo 2, ridicăm soluția modulo 2^2 , apoi rezolvăm sistemul modulo 5653.

Soluția sistemului modulo 2 este: $x_2 \equiv 1 \pmod{2}$, $x_3 \equiv 1 \pmod{2}$, $x_5 \equiv 1 \pmod{2}$ și $x_7 \equiv 1 \pmod{2}$. Pentru a ridica soluția modulo 2^2 , ne folosim de Lema lui Hensel adaptată unui sistem de congruențe (1.3.2). Așadar, calculăm

$$A^{-1} \equiv \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix} \pmod{2}, \quad w_1 = \begin{bmatrix} 1291 \\ 6104 \\ 9073 \\ 6415 \end{bmatrix}$$

și obținem soluția

$$X_2 \equiv \begin{bmatrix} 1 \\ 3 \\ 3 \\ 1 \end{bmatrix} \pmod{4},$$

deci $x_2 \equiv 1 \pmod{4}$, $x_3 \equiv 3 \pmod{4}$, $x_5 \equiv 3 \pmod{4}$ și $x_7 \equiv 1 \pmod{4}$.

Soluția sistemului modulo 5653 este: $x_2 \equiv 8 \pmod{5653}$, $x_3 \equiv 3551 \pmod{5653}$, $x_5 \equiv 2493 \pmod{5653}$ și $x_7 \equiv 1558 \pmod{5653}$. Mai departe, ne folosim de Lema Chineză a

Resturilor 1.2.3 și obținem soluția modulo $p - 1$ a sistemului (S_2):

$$\begin{aligned}\log_g 2 &\equiv 5661 \pmod{22612}, \\ \log_g 3 &\equiv 3551 \pmod{22612}, \\ \log_g 5 &\equiv 13799 \pmod{22612}, \\ \log_g 7 &\equiv 18517 \pmod{22612}.\end{aligned}$$

Pentru a afla soluția problemei logaritmului discret (2.5), trebuie să mai generăm un număr de forma $hg^s \pmod{p}$ care să fie $\mathcal{B}$ -smooth, pentru s arbitrar. După mai multe încercări, am găsit, pentru $s = 11411$, $hg^s \equiv 5625 \equiv 3^2 \cdot 5^4 \pmod{22613}$. Așadar, după ce logaritmăm, avem:

$$\log_g h \equiv 2 \log_g 3 + 4 \log_g 5 - 11411 \pmod{22612}, \quad (2.6)$$

$$\log_g h \equiv 5663 \pmod{22612}. \quad (2.7)$$

Am aflat soluția DLP $\log_g h \equiv 5663 \pmod{22612}$, ce furnizează mulțimea de soluții $S = \{5663 + k \cdot 22612 : k \in \mathbb{Z}\}$.

Capitolul 3

Complexitate

Acest capitol este dedicat studiului complexității metodei indexului, urmărind și detaliind prezentarea din [5]. În prima parte sunt prezentate noțiuni fundamentale despre complexitatea calculului, urmate de o analiză amănunțită a complexității metodei indexului.

3.1 Notatii

Observația 3.1.1. Reamintim că **baza** $\mathcal{B}$ este mulțimea ordonată a numerelor prime mai mici sau egale ca numărul prim P . Notăm:

$$\mathcal{B} = \{2, 3, 5, \dots, P\}.$$

Spunem că un număr întreg x este $\mathcal{B}$ – *smooth* dacă toți factorii primi din descompunerea sa sunt mai mici sau egali ca P .

Notăm **cardinalul mulțimii** $\mathcal{B}$ cu $\pi(P)$, adică numărul total de numere prime mai mici sau egale ca P .

Definiția 3.1.2. Notăm $\psi(X, P)$ numărul de întregi $\mathcal{B}$ – *smooth* n , cu proprietatea că $1 < n \leq X$.

De exemplu,

$$\psi(15, 5) = 10,$$

deoarece $P = 5$, $\mathcal{B} = \{2, 3, 5\}$, iar numerele $\mathcal{B}$ – *smooth* mai mici sau egale ca 15 sunt:

$$2, 3, 4, 5, 6, 8, 9, 10, 12, 15,$$

adică, în total, 10 numere.

Pentru a analiza și compara performanța algoritmilor care rezolvă problema logaritmului discret, trebuie să introducem noțiunea de complexitate.

Complexitatea este o măsură a resurselor (de timp sau memorie) necesare pentru a executa un algoritm. În analiza noastră, vom studia complexitatea în timp, ce depinde de numărul de operații pe care le face algoritmul.

Definiția 3.1.3. (Notăția Big- $\mathcal{O}$) Fie $f : \mathbb{N} \rightarrow \mathbb{R}^+$ o funcție. Notăm cu $T(n)$ timpul de execuție al unui algoritm, unde n reprezintă dimensiunea datelor de intrare. Spunem că $T(n) \in \mathcal{O}(f(n))$ sau $T(n) = \mathcal{O}(f(n))$ dacă există constantele c și n_0 , independente de n , astfel încât:

$$0 \leq T(n) \leq c \cdot f(n), \text{ pentru orice } n \geq n_0.$$

Notăția $\mathcal{O}$ furnizează o limită superioară pentru ordinul timpului de execuție, astfel se poate descrie complexitatea în timp a unui algoritm în cazul cel mai defavorabil.

Definiția 3.1.4. Fie $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$ două funcții. Notăm $f(X) = o(g(X))$ dacă

$$\lim_{X \rightarrow \infty} \frac{f(X)}{g(X)} = 0.$$

În alte cuvinte, $f(n) = o(g(n))$ dacă, pentru orice număr real $c > 0$, există un număr n_0 astfel încât $f(n) < cg(n)$, pentru orice $n > n_0$.

Observația 3.1.5. Notăția $o(1)$ reprezintă o funcție care tinde la 0 atunci când X tinde la ∞ .

În general, dimensiunea datelor de intrare este reprezentată prin numărul de biți pe care îi ocupă în memorie. Presupunem că dimensiunea datelor de intrare este de ordinul $\mathcal{O}(k)$ și există o constantă $A \geq 0$, independentă de aceasta, astfel încât problema poate fi rezolvată în $\mathcal{O}(k^A)$ pași. În acest caz, spunem că problema este rezolvată în *timp polinomial*. Algoritmii polinomiali sunt considerați rapizi.

Pe de altă parte, dacă există o constantă $c > 0$ pentru care algoritmul rezolvă problema în $\mathcal{O}(e^{ck})$ pași, atunci spunem că problema este rezolvată în timp *exponențial*. Algoritmii care rulează în timp exponențial sunt considerați algoritmi lenti.

Un algoritm ce are timp de execuție *subexponențial* este un algoritm a cărui complexitate în timp este mai mică decât exponențială, dar mai mare decât polinomială, atunci când dimensiunea datelor de intrare crește. Pentru date de intrare de $\mathcal{O}(k)$ biți, se notează complexitatea algoritmului subexponențial în timp cu $\mathcal{O}_\epsilon(e^{\epsilon k})$. Acest lucru înseamnă că valorile constante c și n_0 din definiția complexității Big- $\mathcal{O}$ depind de valoarea $\epsilon > 0$.

3.2 Complexitatea metodei indexului

Fie problema logaritmului discret $g^x \equiv h \pmod{p}$, unde p este un număr prim între 2^{k-1} și 2^k , $k \in \mathbb{N}^*$. Așadar, g , h și p au cel mult k biți, deci datele de intrare sunt

de ordinul $\mathcal{O}(k)$. Conform [5], căutarea exhaustivă, cea mai lentă metodă de rezolvare a DLP, are complexitatea în timp $\mathcal{O}(p) = \mathcal{O}(2^k)$, aşadar timp de execuţie exponenţial. Atât algoritmul lui Shanks, cât şi algoritmul Pollard Rho aduc îmbunătăţiri, având o complexitate în timp de $\mathcal{O}(\sqrt{p} \log p)$, respectiv $\mathcal{O}(\sqrt{p})$, însă rămân ineficiente atunci când p este mare. Algoritmul Pohlig-Hellman este rapid atunci când $p - 1$ se descompune în factori primi mici. Vom arăta că **metoda indexului** rezolvă problema logaritmului discret în $\mathcal{O}(e^{c\sqrt{(\ln p)(\ln \ln p)}})$ paşi, deci este un algoritm subexponenţial.

Pentru a arăta acest lucru, vom introduce o serie de rezultate ajutătoare. De asemenea, introducem funcţia $L(X) = e^{\sqrt{\ln X \cdot \ln \ln X}}$, $L : [e, \infty) \rightarrow \mathbb{R}^+$, care va fi foarte utilă în următoarele rezultate.

Teorema 3.2.1 (J. Hadamard, C. J. de la Vallée-Poussin, Teorema numerelor prime). *Fie $\pi(X)$ numărul de numere prime mai mici sau egale ca X . Atunci:*

$$\lim_{X \rightarrow \infty} \frac{\pi(X)}{X / \ln X} = 1.$$

Demonstraţia pentru această teoremă se poate găsi în [2, Chapter 13].

Teorema 3.2.2 (Canfield, Erdős, Pomerance). *[5, Chapter 3.7.1] Fixăm un număr $0 < \epsilon < \frac{1}{2}$, şi presupunem că X şi P cresc împreună astfel încât*

$$(\ln X)^\epsilon < \ln P < (\ln X)^{1-\epsilon}.$$

Notăm

$$u = \frac{\ln X}{\ln P}.$$

Atunci, numărul de numere $\mathcal{B}$ -smooth mai mici decât X satisface relaţia

$$\psi(X, P) = X \cdot u^{-u(1+o(1))}.$$

Demonstraţia depăşeşte scopul lucrării, însă poate fi găsită în [3].

Corolarul 3.2.3. *Pentru orice număr c fixat, cu $0 < c < 1$,*

$$\lim_{X \rightarrow \infty} \psi(X, L(X)^c) = X \cdot L(X)^{-\frac{1}{2c}(1+o(1))} \quad .$$

Demonstraţie. Fie $P = L(X)^c$ şi $0 < \epsilon < \frac{1}{2}$. Avem:

$$\ln P = \ln L(X)^c = c \ln L(X) = c\sqrt{\ln X \ln \ln X}$$

Verificăm că este satisfăcută relația:

$$(\ln X)^\epsilon < \ln P < (\ln X)^{1-\epsilon} \quad (3.1)$$

$$\iff (\ln X)^\epsilon < c \cdot (\ln X)^{\frac{1}{2}} (\ln \ln X)^{\frac{1}{2}} < (\ln X)^{1-\epsilon}. \quad (3.2)$$

Verificăm prima inegalitate: $(\ln X)^\epsilon < c \cdot (\ln X)^{\frac{1}{2}} (\ln \ln X)^{\frac{1}{2}}$:

Pentru X mare, $\ln X > 0$, deci putem împărți inecuația la $(\ln X)^\epsilon$ și obținem:

$$1 < c \cdot (\ln X)^{\frac{1}{2}-\epsilon} (\ln \ln X)^{\frac{1}{2}}.$$

Cum $0 < \epsilon < \frac{1}{2}$, avem $\frac{1}{2} - \epsilon > 0$ și $\lim_{X \rightarrow \infty} c \cdot (\ln X)^{\frac{1}{2}-\epsilon} (\ln \ln X)^{\frac{1}{2}} = \infty$, inegalitatea este adevărată.

Verificăm a doua inegalitate: $c \cdot (\ln X)^{\frac{1}{2}} (\ln \ln X)^{\frac{1}{2}} < (\ln X)^{1-\epsilon}$:

Cum, pentru X mare, toți termenii din inegalitate sunt pozitivi, putem logaritma inegalitățile (3.1) și obținem:

$$\epsilon(\ln \ln X) < \ln c + \frac{1}{2} \ln \ln X + \frac{1}{2} \ln \ln \ln X < (1 - \epsilon) \ln \ln X.$$

Notăm $\ln \ln \ln X = t$, deci $\ln \ln X = e^t$ și obținem:

$$\ln c + \frac{1}{2} e^t + \frac{1}{2} t < (1 - \epsilon) e^t \iff e^t \left(\epsilon - \frac{1}{2} \right) + \frac{1}{2} t < -\ln c.$$

Pentru $0 < \epsilon < \frac{1}{2}$ avem $\epsilon - \frac{1}{2} < 0$ și cum $0 < c < 1$, $\ln c < 0$, deci $-\ln c > 0$, rezultă că inegalitatea este adevărată pentru t mare, deoarece funcția exponențială e^t crește mai repede decât orice polinom și $\lim_{t \rightarrow \infty} e^t \left(\epsilon - \frac{1}{2} \right) + \frac{1}{2} t = -\infty$. Așadar, putem aplica Teorema 3.2.2 cu:

$$u = \frac{\ln X}{\ln P} = \frac{\ln X}{c \sqrt{\ln X \ln \ln X}} = \frac{1}{c} \sqrt{\frac{\ln X}{\ln \ln X}}$$

și obținem $\psi(X, P) = X \cdot u^{-u(1+o(1))}$, unde

$$u^{-u(1+o(1))} = \left(\frac{1}{c} \sqrt{\frac{\ln X}{\ln \ln X}} \right)^{-\frac{1}{c} \sqrt{\frac{\ln X}{\ln \ln X}} (1+o(1))} \quad \text{și} \quad L(X) = e^{\sqrt{\ln X \cdot \ln \ln X}}.$$

Arătăm că: $u^{-u(1+o(1))} = L(X)^{-\frac{1}{2c}(1+o(1))}$:

$$\begin{aligned}
\ln(u^{-u}) &= -u \ln u = \\
&= -\left(\frac{1}{c} \sqrt{\frac{\ln X}{\ln \ln X}}\right) \ln \left(\frac{1}{c} \sqrt{\frac{\ln X}{\ln \ln X}}\right) = \\
&= -\left(\frac{1}{c} \sqrt{\frac{\ln X}{\ln \ln X}}\right) \left(\ln \frac{1}{c} + \frac{1}{2} \ln \ln X - \frac{1}{2} \ln \ln \ln X\right) = \\
&= -\frac{1}{c} \sqrt{\frac{\ln X}{\ln \ln X}} \ln \frac{1}{c} - \frac{1}{2c} \sqrt{\ln X \ln \ln X} + \frac{1}{2c} \sqrt{\frac{\ln X}{\ln \ln X}} \ln \ln \ln X.
\end{aligned}$$

Căutăm termenul dominant din această expresie. Cum $\ln \ln \ln X \ll \ln \ln X$ pentru X mare, rezultă că:

$$\sqrt{\frac{\ln X}{\ln \ln X}} \ln \ln \ln X \ll \sqrt{\frac{\ln X}{\ln \ln X}} \ln \ln X = \sqrt{\ln X \ln \ln X}.$$

Așadar, cand $X \rightarrow \infty$, termenul dominant este $-\frac{1}{2c} \sqrt{\ln X \ln \ln X} = -\frac{1}{2c} \ln L(X)$, deci putem aproxima $\ln(u^{-u}) \approx -\frac{1}{2c} \ln L(X)$ și $u^{-u} \approx L(X)^{-\frac{1}{2c}}$.

Cu aceste aproximări făcute, obținem $u^{-u(1+o(1))} = L(X)^{-\frac{1}{2c}(1+o(1))}$ pentru X mare și:

$$\lim_{X \rightarrow \infty} \psi(X, L(X)^c) = X \cdot L(X)^{-\frac{1}{2c}(1+o(1))}.$$

□

Propoziția 3.2.1. Fie $L(X) = e^{\sqrt{\ln X \cdot \ln \ln X}}$, N un număr întreg mare și alegem $P = L(N)^{1/\sqrt{2}}$, unde N și P respectă condițiile Teoremei 3.2.2. Ne așteptăm să testăm aproximativ $L(N)^{\sqrt{2}}$ numere alese în mod aleator modulo N astfel încât să găsim $\pi(P)$ numere $\mathcal{B}$ -smooth.

Demonstrație. Utilizăm definiția probabilității:

$$p = \frac{\text{număr cazuri favorabile}}{\text{număr cazuri posibile}}.$$

Probabilitatea de a alege în mod aleator un număr modulo N care să fie $\mathcal{B}$ -smooth este:

$$\frac{\psi(N, P)}{N}.$$

De asemenea, utilizând același raționament, probabilitatea de succes aferentă unei încercări individuale este egală cu raportul dintre numărul de reușite și numărul de încercări.

Așadar, pentru a găsi $\pi(P)$ numere $\mathcal{B}$ – *smooth*, trebuie să testăm aproximativ

$$\frac{\pi(P)}{\left(\frac{\psi(N, P)}{N}\right)} \quad (3.3)$$

numere.

Pentru a avea un algoritm cât mai eficient, vrem să testăm cât mai puține numere, deci căutăm numărul P care să minimizeze raportul (3.3).

Corolarul 3.2.3 afirmă că, pentru N mare și $0 < c < 1$, putem face următoarea aproximare:

$$\frac{\psi(N, L(N)^c)}{N} \approx L(N)^{-\frac{1}{2c}}.$$

Așadar, luăm pe $P = L(N)^c$ și căutăm pe c care să minimizeze raportul (3.3).

De asemenea, din Teorema 3.2.1, putem aproxima numărul de numere prime mai mici sau egale cu P astfel:

$$\pi(P) \approx \frac{P}{\ln P}.$$

Cu aceste aproximări, raportul (3.3) devine:

$$\frac{\pi(L(N)^c)}{\frac{\psi(N, L(N)^c)}{N}} \approx \frac{L(N)^c}{c \ln L(N)} \cdot \frac{1}{L(N)^{-\frac{1}{2c}}} = L(N)^{c+\frac{1}{2c}} \cdot \frac{1}{c \ln L(N)}. \quad (3.4)$$

Factorul care domină ecuația 3.4 este $L(N)^{c+\frac{1}{2c}}$, deci alegem numărul c care minimizează $c + \frac{1}{2c}$.

Fie $f(x) = x + \frac{1}{2x}$, unde $x > 0$. Avem $f'(x) = 1 - \frac{1}{2x^2}$, deci $f'(x) = 0 \iff x = \pm \frac{1}{\sqrt{2}}$. Dar $x > 0$, deci alegem $x = \frac{1}{\sqrt{2}}$. Astfel, $f(x) \geq f\left(\frac{1}{\sqrt{2}}\right) = \sqrt{2}$.

Așadar, luăm $c = \frac{1}{\sqrt{2}}$ și $c + \frac{1}{2c} = \sqrt{2}$.

În concluzie, dacă alegem $P \approx L(N)^{\frac{1}{\sqrt{2}}}$, va trebui să testăm aproximativ $L(N)^{\sqrt{2}}$ numere pentru a găsi $\pi(P)$ numere $\mathcal{B}$ – *smooth*. □

Reamintim că metoda indexului cuprinde două etape, motiv pentru care analiza complexității algoritmului trebuie realizată separat pentru fiecare dintre acestea.

Prima etapă presupune găsirea a $\pi(P)$ numere $\mathcal{B}$ – *smooth* de forma $g^{t_i} \bmod p$, ce sunt folosite ulterior pentru crearea unui sistem de congruențe modulo $p - 1$ compatibil determinat, utilizat pentru a calcula logaritmiile elementelor din bază. Așadar, conform Propoziției 3.2.1, unde $N = p$, pentru a avea un algoritm optim, trebuie să alegem baza $\mathcal{B}$ astfel încât cel mai mare număr prim din bază să fie aproximativ $L(p)^{\frac{1}{\sqrt{2}}}$. Cu această bază, ne așteptăm să testăm $L(p)^{\sqrt{2}}$ numere pentru a putea alcătui sistemul de congruențe. Așadar, procesul prin care calculăm logaritmiile elementelor din bază are o complexitate în timp de ordinul $\mathcal{O}(e^{(\sqrt{2}+o(1))\sqrt{(\ln p)(\ln \ln p)}})$.

Observația 3.2.4. În demonstrația Propoziției 3.2.1, am aproximat

$$\frac{\psi(N, L(N)^c)}{N} \approx L(N)^{-\frac{1}{2c}},$$

ignorând termenul $(1 + o(1))$ de la exponent din Corolarul 3.2.3, pentru a putea găsi un număr c exact. Ținem totuși cont de acest aspect atunci când analizăm complexitatea algoritmului și adăugăm acest termen la final, în expresia $\mathcal{O}(e^{(\sqrt{2}+o(1))\sqrt{(\ln p)(\ln \ln p)}})$.

Observația 3.2.5. Există metode de optimizare în implementarea algoritmului ce reușesc să scadă numărul de teste necesare primei etape a metodei indexului până la o mică putere a lui $L_{\frac{1}{3}}(p)$, unde:

$$L_n(X) = e^{(\ln X)^n (\ln \ln X)^{1-n}}.$$

Utilizând varianta optimizată a algoritmului prezentat în Capitolul 4.2.2, am reușit o performanță similară. De această dată, am actualizat baza de numere prime la fiecare rulare, astfel încât am ales P , cel mai mare element din bază, ca fiind cel mai apropiat număr prim de $L(p)^{\frac{1}{\sqrt{2}}}$. Am rulat algoritmul pentru metoda indexului de câte 1000 de ori pentru fiecare set de date inițiale (valorile p , g și h) și am calculat la fiecare iterație câte numere au fost testate pentru a fi $\mathcal{B} - smooth$ în prima etapă a metodei indexului. Apoi, am calculat media acestor valori pentru a avea o aproximare cât mai corectă și am comparat această valoare cu $L_{\frac{1}{3}}(p)$, $L_{\frac{1}{3}}(p)^{\sqrt{2}}$ și $L(p)^{\sqrt{2}}$. Datele pot fi vizualizate în tabelul următor:

Date inițiale	Nr. P din bază	Media	$L_{\frac{1}{3}}(p)$	$L_{\frac{1}{3}}(p)^{\sqrt{2}}$	$L(p)^{\sqrt{2}}$
$p = 18443, g = 37, h = 211$	29	168,234	41	190	812
$p = 100003, g = 2, h = 68005$	43	360,649	60	327	1810
$p = 319993, g = 5, h = 294443$	53	556,625	77	465	3054
$p = 569843, g = 2, h = 200145$	61	686,03	86	544	3930
$p = 611953, g = 7, h = 78902$	61	696,93	87	553	4053
$p = 1034317, g = 2, h = 735679$	71	873,401	97	645	5075

Tabelul 3.1: Tabel complexitate generare numere $\mathcal{B} - smooth$

Putem observa din Tabelul 3.1 că media numărului de teste efectuate (datele situate pe coloana *Media*) este apropiată de valoarea rotunjită a lui $L_{\frac{1}{3}}(p)^{\sqrt{2}}$, ce denotă o performanță bună a algoritmului. De asemenea, comparând numărul mediu de teste efectuate cu aproximarea complexității teoretice a primei etape a metodei indexului ($L(p)^{\sqrt{2}}$), putem observa că algoritmul implementat este de până la șase ori mai rapid.

Pentru a doua etapă a metodei indexului, complexitatea în timp este dictată de găsirea unui singur număr $\mathcal{B} - smooth$ de forma $h \cdot g^s \bmod p$, cu ajutorul căruia calculăm logaritmul discret $\log_g h \bmod p - 1$. Utilizând calcule similare celor din demonstrația Propoziției 3.2.1, trebuie testate aproximativ $L(p)^{\frac{\sqrt{2}}{2}}$ numere.

Rămâne de demonstrat că metoda indexului are o complexitate în timp subexponențială. Pentru acest lucru introducem o nouă notație:

$$L_N[\alpha, c] := e^{(c+o(1))(\ln N)^\alpha (\ln \ln N)^{1-\alpha}}.$$

Pentru $N = p$, observăm că:

$$L_p[0, c] = e^{(c+o(1)) \ln \ln p} = (\ln p)^{c+o(1)}$$

este polinomială în timp în $\ln p$, în timp ce:

$$L_p[1, c] = e^{(c+o(1)) \ln p} = p^{c+o(1)}$$

este exponențială în $\ln p$. Pentru $0 < \alpha < 1$, $L_p[\alpha, c]$ este subexponențială (în $\ln p$).

Metoda indexului presupune aproximativ $L_p[\frac{1}{2}, c]$ numere testate, unde $c = \sqrt{2}$ în prima etapă, respectiv $c = \frac{\sqrt{2}}{2}$ în cea de-a doua etapă, așadar este subexponențială în timp.

Capitolul 4

Implementare

Am dezvoltat un algoritm în limbajul de programare **Python** care aplică metoda indexului pentru problema logaritmului discret pe un set de date inițiale introduse de la tastatură. Toate datele din tabelele prezente în această lucrare au fost obținute prin utilizarea acestei implementări.

Vom parcurge etapele esențiale ale algoritmului, însă, mai întâi, vom defini funcțiile cele mai importante care sunt utilizate în funcția principală. Funcțiile mai puțin importante vor fi doar explicate în această lucrare. Codul complet este disponibil online, accesând [10].

De asemenea, două variante ale algoritmului au fost dezvoltate și vor fi prezentate în detaliu în subcapitolele următoare. În plus, am realizat o analiză comparativă a acestora pentru studiul eficienței lor.

Este important de menționat că au fost folosite librării din Python precum: **math**, pentru calcul de radicali, **numpy**, pentru calcule matriceale, cum ar fi determinantul unei matrice și **random**, pentru generarea de numere în mod aleator dintr-un interval.

4.1 Funcții ajutătoare

În cadrul metodei indexului, de multe ori sunt necesare operații de ridicare la putere efectuate modulo un număr întreg. Aceste operații sunt predefinite în Python, însă sunt costisitoare ca timp. De aceea, am creat funcția **exponentiere**(x, e, p) care are ca parametri de intrare numărul întreg x , puterea la care vrem să-l ridicăm pe x , e , și numărul întreg p , ce implică reducerea modulo p . Funcția se bazează pe algoritmul de exponențiere rapidă și pe funcția $\varphi(x)$ a lui Euler (1.1.2) și poate fi utilizată atât pentru ridicări la puteri pozitive, cât și negative (atunci când este posibilă această operație).

Algoritmul 1 (Algoritmul de exponențiere rapidă). [5, Chapter 1.3.2]

Avem date g, a, n trei numere întregi nenule și vrem să calculăm $g^a \bmod n$.

1. Calculăm descompunerea binară a lui a :

$$a = a_0 + a_1 \cdot 2 + a_2 \cdot 2^2 + \dots + a_r \cdot 2^r,$$

unde $a_0, \dots, a_{r-1} \in \{0, 1\}$ și $a_r = 1$.

2. Calculăm puterile $g^{2^i} \bmod n$ pentru $0 \leq i \leq r$ prin ridicare succesivă la pătrat:

$$\begin{aligned} t_0 &\equiv g \bmod n, \\ t_1 &\equiv t_0^2 \equiv g^2 \bmod n, \\ t_2 &\equiv t_1^2 \equiv g^{2^2} \bmod n, \\ &\vdots \\ t_r &\equiv t_{r-1}^2 \equiv g^{2^r} \bmod n. \end{aligned}$$

3. Calculăm $g^a \bmod n$ folosind formula

$$g^a \equiv g^{a_0 + a_1 \cdot 2 + a_2 \cdot 2^2 + \dots + a_r \cdot 2^r} \equiv g^{a_0} (g^2)^{a_1} (g^{2^2})^{a_2} \dots (g^{2^r})^{a_r} \equiv t_0^{a_0} t_1^{a_1} t_2^{a_2} \dots t_r^{a_r} \bmod n.$$

Observăm că valorile $t_0, t_1, \dots, t_r$ au fost deja calculate la pasul 2. Astfel, produsul de mai sus se obține prin selectarea tuturor factorilor t_i pentru care exponentul a_i este egal cu 1, apoi înmulțirea lor.

În cazul în care exponentul e este pozitiv, funcția **exponentiere**(x, e, p) aplică algoritmul de exponențiere rapidă prezentat mai sus și returnează $x^e \bmod p$.

În cazul în care exponentul e este negativ, funcția **exponentiere**(x, e, p) calculează mai întâi inversul modulo p al lui x (dacă acest lucru este posibil, adică dacă $\text{cmmdc}(x, p) = 1$), iar apoi aplică algoritmul de exponențiere rapidă pentru a ridica inversul lui x modulo p la valoarea absolută a exponentului e .

Calculul inversului modulo p al lui x se face utilizând o consecință imediată a Teoremei lui Euler 1.1.4:

$$x^{-1} \equiv x^{\varphi(p)-1} \bmod p, \quad \text{pentru orice } x \in \mathbb{Z}, \quad \text{cu } \text{cmmdc}(x, p) = 1.$$

Pentru calculul $\varphi(p)$ este necesară factorizarea lui p . Așadar, calculăm descompunerea în factori primi a lui p prin intermediul funcției **Factorizare**(p), care returnează un dicționar ce are ca și chei factorii primi din descompunerea lui p și ca valori exponenții corespunzătorii fiecărui factor prim în parte. Cu aceste informații, putem calcula inversul lui x modulo p .

```

1 def exponentiere(x, e, p):
2     if x % p == 0:
3         if e < 0:

```

```

4         print(f"Nu exista invers modular pentru {x} mod {p}.")
5         return None
6     else:
7         return 0
8 if e < 0:
9     if cmmdc(x, p) == 1:
10         phi = p
11         factorizare_p = Factorizare(p)
12         for i in factorizare_p.keys():
13             phi = phi * (i - 1) // i
14         e_nou = phi - 1
15         nr = exponentiere(x, e_nou, p)
16         nr = exponentiere(nr, np.abs(e), p)
17         return nr
18     else:
19         print(f"Nu exista invers modular pentru {x} mod {p}.")
20         return None
21 # Daca x diferit de 0 mod p si e este pozitiv:
22 r = e % 2
23 e = e // 2
24 exp = x # x^(2^i)
25 nr = 1
26 if r == 1:
27     nr = exp
28     nr = nr % p
29 while e > 0:
30     r = e % 2
31     e = e // 2
32     exp = exp**2
33     exp = exp % p
34     if r == 1:
35         nr = nr * exp
36         nr = nr % p
37 return nr

```

4.1: Funcție exponențiere modulară

O altă funcție esențială în algoritm este **B_smooth**($\mathcal{B}, g_i$) care are ca date de intrare baza $\mathcal{B}$ de numere prime utilizată în algoritm și numărul g_i care este testat pentru a fi $\mathcal{B}$ – *smooth*. Dacă numărul g_i este $\mathcal{B}$ – *smooth*, funcția returnează valoarea *True* și descompunerea în factori primi a numărului g_i sub forma unui dicționar, ce are o structură identică cu cea a dicționarului returnat de funcția **Factorizare**(g_i). În caz

contrar, returnează *False* și *None*.

Funcția acționează pe același principiu ca ciurul lui Eratostene. Pentru fiecare prim p din baza $\mathcal{B}$, testează dacă îl divide pe g_i și îl împarte pe g_i la p de numărul maxim de ori posibil. După acest proces repetat pentru fiecare număr prim din baza, dacă în urma împărțirilor repetate rămânem cu valoarea 1, atunci numărul g_i este $\mathcal{B}$ – *smooth*.

Exemplul 4.1.1. Dacă avem baza de numere prime $\mathcal{B} = \{2, 3, 5\}$ și numărul 60 pe care vrem să-l verificăm dacă este $\mathcal{B}$ – *smooth*, vom efectua următoarele împărțiri:

$$\begin{aligned} 60 : 2 &= 30 \\ 30 : 2 &= 15 \\ 2 \nmid 15, \quad &\text{așadar trecem la următorul prim din bază} \\ 15 : 3 &= 5 \\ 3 \nmid 5, \quad &\text{așadar trecem la următorul prim din bază} \\ 5 : 5 &= 1. \end{aligned}$$

Cum am obținut la final 1, numărul 60 este $\mathcal{B}$ – *smooth*.

```
1 def B_smooth(B, gi):
2     d = {}
3     for p in B:
4         if gi % p == 0:
5             putere = 1
6             gi = gi // p
7             r = 0
8             while r == 0:
9                 r = gi % p
10                if r == 0:
11                    putere += 1
12                    gi = gi // p
13                else:
14                    r = 1
15                d[p] = putere
16            else:
17                d[p] = 0
18    if gi == 1:
19        return True, d
20    else:
21        return False, None
```

4.2: Funcție verificare B-smooth

4.2 Funcția principală

Funcția principală **Metoda_Indexului**(p, g, h) are ca date de intrare 3 numere întregi p, g și h și returnează soluția unică modulo $p - 1$ a problemei logaritmului discret $g^x \equiv h \pmod{p}$, în cazul în care sunt îndeplinite condițiile inițiale ale problemei. În caz contrar, returnează *None*. Prima dată sunt verificate: primalitatea lui p , iar g trebuie să fie generator în grupul ciclic $(\mathbb{Z}_p^*, \cdot)$. Dacă aceste condiții sunt îndeplinite, ne asigurăm că g și h sunt în intervalul $[0, p - 1]$ și apoi putem trece la pasul de construire a bazei $\mathcal{B}$ de numere prime, conform specificațiilor din Propoziția 3.2.1. Pentru a construi baza într-un mod optim, calculăm mai întâi valoarea $L(p)^{1/\sqrt{2}}$ și adăugăm apoi în bază toate numerele prime mai mici sau egale cu această valoare.

Prima etapă a metodei indexului constă în aflarea logaritmilor din elementele bazei de numere prime, adică $\log_g p_i$, $i \in \{1, \dots, B = |\mathcal{B}|\}$. Pentru a obține soluție unică, trebuie să rezolvăm sistemul S_2 alcătuit din B congruențe, care să fie compatibil, unic determinat modulo $p - 1$. Acest calcul este realizat de funcția **etapa1**($p, g, h, \mathcal{B}$), ce returnează o listă în care sunt $\log_g p_i$, $i \in \{1, \dots, B\}$.

Am dezvoltat **două variante de construire și rezolvare a acestui sistem**, ce vor fi detaliate în următoarele subcapitole.

Cea de-a doua etapă presupune aflarea soluției $\log_g h \pmod{p - 1}$. Pentru a obține soluția căutată, trebuie să generăm un nou număr $\mathcal{B} - smooth$ de forma $h \cdot g^s \pmod{p}$, unde s este un exponent ales în mod aleator în intervalul $[1, p - 2]$, cu ajutorul funcției **random.randint**($1, p - 2$). Variabila nr stochează valoarea $h \cdot g^s \pmod{p}$. Aplicăm funcția **B_smooth**($\mathcal{B}, nr$) pentru a verifica proprietatea necesară și pentru a obține factorizarea numărului nr . În cazul în care nr este $\mathcal{B} - smooth$, continuăm cu calculele prezentate în Capitolul 2.3 și returnăm rezultatul.

```
1 def Metoda_Indexului(p, g, h):
2     if not verificare_prim(p):
3         print(f'p = {p} nu este prim.')
4         return None
5     if not verificare_elem_primitiv(g, p):
6         print(f'Elementul g = {g} nu este primitiv.')
7         return None
8     g = g % p
9     h = h % p
10    b = round(L(p)**(1/math.sqrt(2)))
11    B = []
12    for i in range(b + 1):
```

```

13         if verificare_prim(i):
14             B.append(i)
15     # ETAPA 1: Aflam log_g(pi), unde pi sunt elementele din baza B
16     logaritmi_baza = etapa1(p, g, h, B)
17     # ETAPA 2: Aflarea logaritmului cautat log_g(h)
18     count = 0
19     while count < 1:
20         s = random.randint(1, p-2)
21         gs = exponentiere(g, s, p)
22         nr = (h * gs) % p
23         valoare_adevar, factorizare = B_smooth(B, nr)
24         if valoare_adevar:
25             count = 1
26             exponenti = np.zeros(len(B)).astype(int)
27             for i in factorizare.keys():
28                 for prim in B:
29                     if i == prim:
30                         exponenti[B.index(prim)] = factorizare[i]
31     log = -s
32     for i in range(len(B)):
33         log = log + exponenti[i] * logaritmi_baza[i]
34     log = log % (p-1)
35     print("Solutia din  $\mathbb{Z}_{p-1}$  a problemei logaritmului discret  $g^x = h$ 
36           mod  $p$  este: ", log)
37     return log

```

4.3: Funcția principală

4.2.1 Prima variantă de alcătuire a sistemului de congruențe

Prima variantă constă în generarea succesivă a câte $C = B$, numere $\mathcal{B}$ – *smooth* care să alcătuiască sistemul de congruențe pătratic S_2 , precum este prezentat în Capitolul 2.2. Prin intermediul structurii *while* creăm un proces iterativ ce se oprește atunci când s-a găsit un sistem de congruențe compatibil determinat, adică un sistem a cărui matrice are determinantul inversabil modulo $p - 1$. Așadar, este posibilă extragerea soluției unice modulo $p - 1$.

Deoarece $p - 1$ nu este un număr prim, îl vom descompune în factori primi și apoi vom rezolva sistemul de congruențe modulo fiecare factor prim în parte. Deoarece ne-am asigurat că matricea sistemului are determinantul inversabil modulo $p - 1$, atunci el o să fie, automat, inversabil modulo fiecare factor prim din descompunerea lui $p - 1$ și putem aplica metoda lui Gauss.

În rezolvare, trebuie să ținem cont de puterea la care se găsește fiecare factor prim. Dacă unul dintre aceștia se găsește la o putere mai mare decât 1, atunci vom folosi Lema lui Hensel aplicată pentru sisteme de congruențe (1.3.2) pentru a ridica soluția modulo factorul respectiv la puterea corespunzătoare. La final, se găsește soluția unică modulo $p - 1$ (adică soluția sistemului S_2) prin aplicarea Lemei Chineze a Resturilor 1.2.3.

Codul pentru acest algoritm se poate găsi în Anexa A.

4.2.2 A doua variantă de alcătuire a sistemului de congruențe

A doua variantă implementată începe prin generarea a B numere $\mathcal{B}$ – *smooth*, ce determină un sistem pătratic de congruențe modulo $p - 1$. Spre deosebire de prima variantă, unde mai întâi căutam un sistem de congruențe care să fie compatibil determinat modulo $p - 1$, și apoi calculăm soluțiile, varianta a doua face aceste două procese concomitent.

Plecând de la sistemul nostru inițial pătratic, aplicăm metoda Gauss-Jordan modulo fiecare factor prim din descompunerea lui $p - 1$. Indiferent de puterea la care se găsesc în descompunere, considerăm fiecare factor inițial la puterea 1.

Fie factorizarea lui $p - 1$:

$$p - 1 = q_1^{\alpha_1} \cdot q_2^{\alpha_2} \cdot \dots \cdot q_m^{\alpha_m},$$

unde $q_1, \dots, q_m \in \mathbb{N}^*$ sunt numere prime distincte, iar $\alpha_1, \alpha_2, \dots, \alpha_m \in \mathbb{N}^*$.

Începem prin a aplica metoda Gauss-Jordan modulo q_1 . În matricea extinsă a sistemului de congruențe, vrem să găsim câte un pivot pe fiecare coloană, mai puțin ultima, corespunzătoare termenilor liberi. Așadar, vrem să găsim câte un element inversabil modulo q_1 pe fiecare coloană. Dacă am găsit și se află pe altă linie decât pe cea corespunzătoare indexului coloanei curente, permutăm liniile între ele pentru a-l aduce pe poziția corectă. Dacă pivotul se află pe linia corespunzătoare indicelui coloanei curente, atunci putem trece la următorul pas, și anume înmulțim linia pivotului cu inversul modulo q_1 al acestuia, pentru a-l transforma în 1 mod q_1 .

Pentru a nu pierde sistemul inițial de congruențe, vom păstra mereu o copie a acestuia pe care o vom actualiza după cum urmează să explic. Mereu când interschimbăm linii în matricea sistemului modulo factorul prim curent, vom face aceeași interschimbare și în copia matricei, dacă adăugăm o nouă linie, o vom face în ambele matrici, însă nu vom aplica matricei copie operațiile de înmulțire cu scalari, adunare sau scădere de linii/coloane.

Odată ce am obținut 1 mod q_1 pe poziția pivotului, efectuăm operații pe linii pentru a obține zerouri sub și deasupra pivotului, pe celelalte linii ale matricei.

În cazul în care nu am găsit pivot în coloana curentă, vom căuta un număr $\mathcal{B}$ – *smooth* care să genereze o nouă ecuație modulo $p - 1$. Adăugăm această linie atât la matricea modulo q_1 , cât și la matricea copie a sistemului. Aceasta va fi ultima linie în ambele matrici, iar, prin această modificare, matricile nu vor mai fi pătratice, cele două sisteme

având mai multe ecuații decât necunoscute.

Aplicăm ultimei linii din matricea modulo q_1 transformările necesare pentru a obține 0 mod q_1 pe toate coloanele pe care există pivoți. Căutăm pivot pe ultima linie adăugată, pe coloana pe care anterior nu găsisem niciun pivot. Dacă găsim pe acea poziție pivot, atunci putem continua procedeul așa cum a fost descris mai sus. Dacă nici de această dată nu găsim pivot, atunci va trebui să adăugăm, succesiv, câte o linie nouă în matrice, prin același proces descris anterior, până găsim un pivot.

Astfel, este un proces iterativ ce se repetă până când obținem pivoți pe toate coloanele, exceptând-o pe ultima.

Din această formă eșalon extragem soluțiile ce se găsesc pe primele B linii ale ultimei coloane. Așadar, am aflat $\log_g p_i \bmod q_1$, $i \in \{1, \dots, B\}$. De asemenea, dacă $\alpha_1 > 1$, atunci aplicăm Lema lui Hensel 1.3.1 pentru a ridica soluția modulo $q_1^{\alpha_1}$.

Este important de reținut că matricea copie s-a modificat de-a lungul acestui proces, așadar, când trecem la următorul pas (adică la următorul q_k , $k \in \{2, \dots, m\}$), vom lucra cu ea pornind de la cea mai recentă formă, și nu cu matricea pătratică inițială.

Prin urmare, repetăm acest algoritm pentru a afla soluția sistemului de congruențe modulo q_k , pentru $k = 2, \dots, m$. A doua metodă se încheie analog primei metode, prin generarea soluției unice modulo $p - 1$ a logaritmilor elementelor din bază, prin Lema Chineză a Resturilor 1.2.3.

Observația 4.2.1. Pe măsură ce k crește, matricea sistemului este probabil să aibă din ce în ce mai multe linii, ceea ce înseamnă, totodată, că șansele de a găsi pivoți pe fiecare coloană sunt din ce în ce mai mari. Astfel, avem mai puține numere $\mathcal{B}$ -smooth generate și, deci, o eficiență mai mare în execuție.

Codul pentru această variantă de implementare se poate găsi în Anexa B.

4.3 Analiza comparativă a celor două variante de implementare

Pentru a testa eficiența celor două variante de implementare, am dezvoltat o analiză comparativă a celor doi algoritmi în funcție de numărul de numere testate pentru a fi $\mathcal{B}$ -smooth în prima etapă a metodei indexului din fiecare algoritm, întrucât găsirea de numere $\mathcal{B}$ -smooth este una dintre dificultățile întâlnite în implementarea celor două metode.

Pentru a realiza această numărătoare, am inițializat o variabilă *contor* cu 0, după care am actualizat-o la fiecare căutare de noi numere $\mathcal{B}$ -smooth. Am rulat fiecare algoritm pentru același set de date inițiale (p , g și h) și aceeași bază $\mathcal{B} = \{2, 3, 5, 7\}$ de 1000 de ori și am stocat la fiecare iterație valoarea finală a variabilei *contor* într-o listă.

Deoarece căutarea de numere $\mathcal{B}$ – *smooth* presupune generarea de numere aleatoare, rezultatele pot varia semnificativ de la o iterație la alta. Din acest motiv, pentru a obține rezultate cât mai concludente, am comparat mediile (notate cu μ) elementelor din cele doua liste corespunzătoare celor doi algoritmi, alături de alți factori importanți precum deviația standard (pe care o vom nota σ), mediana, minimul și maximul. De asemenea, vom analiza și raportul dintre deviația standard și medie. Toate aceste rezultate sunt surprinse în Tabelele 4.1 și 4.2 de mai jos.

În analiza statistică, am considerat lista de numere de la fiecare metodă eșantionul de date observate, unde fiecare observație x_i , $i = 1, \dots, 1000$ corespunde valorii contorului de la iterația respectivă. Am calculat **media** eșantionului după formula:

$$\mu = \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad \text{pentru } n = 1000,$$

iar **deviația standard** a eșantionului, adică rădăcina pătrată din varianța eșantionului, am calculat-o conform formulei:

$$\sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}, \quad \text{pentru } n = 1000.$$

În ceea ce privește mediana, am ordonat mai întâi lista de numere în mod crescător. Apoi, cum $n = 1000$ este par, am ales mediana ca fiind media aritmetică a elementelor de pe pozițiile $\frac{n}{2} = 500$ și $\frac{n}{2} + 1 = 501$.

Date inițiale	μ	σ	σ/μ	Mediană	Minim	Maxim
$p = 673, g = 5, h = 211$	317.23	320.29	1.01	215	6	2478
$p = 18443, g = 37, h = 211$	850.22	808.07	0.95	612	22	7119
$p = 65537, g = 11, h = 35879$	1878.53	1741.41	0.93	1323	30	17396
$p = 100003, g = 2, h = 68005$	6027.73	5956.20	0.99	4254	159	47967
$p = 319993, g = 5, h = 294443$	11571.10	11350.55	0.98	7879	155	70957
$p = 569843, g = 2, h = 200145$	12304.33	11858.13	0.96	8470	441	81273
$p = 611953, g = 7, h = 78902$	21854.73	20490.14	0.94	16146	548	130745
$p = 1034317, g = 2, h = 73567$	26869.95	25747.83	0.96	19337	617	201874

Tabelul 4.1: Tabel date pentru Varianta 1

4.3.1 Interpretarea rezultatelor

Prin analiza comparativă a Tabelelor 4.1 și 4.2 este evident faptul că **a doua variantă de implementare este mult mai eficientă decât prima variantă**.

În primul rând, numărul mediu de teste corespunzător Variantei 2 (coloana μ din tabel) este semnificativ mai mic decât numărul mediu de teste obținut prin Varianta 1, cu valori de până la cinci ori mai mici. Mai puține numere testate, în medie, implică un

Date inițiale	μ	σ	σ/μ	Mediană	Minim	Maxim
$p = 673, g = 5, h = 211$	50.16	26.88	0.54	45	6	157
$p = 18443, g = 37, h = 211$	296.98	166.72	0.56	261	21	1052
$p = 65537, g = 11, h = 35879$	722.16	399.47	0.55	641	56	2824
$p = 100003, g = 2, h = 68005$	1135.58	582.36	0.51	1020	137	3626
$p = 319993, g = 5, h = 294443$	2523.18	1339.24	0.53	2240	206	9830
$p = 569843, g = 2, h = 200145$	3549.54	1943.29	0.55	3220	116	21037
$p = 611953, g = 7, h = 78902$	4171.77	2232.19	0.53	3789	594	18093
$p = 1034317, g = 2, h = 73567$	5777.08	3025.24	0.52	5309	447	18455

Tabelul 4.2: Tabel date pentru Varianta 2

algorithm mai rapid și, deci, mai eficient.

Un alt element statistic important este mediana, ce reprezintă acea valoarea din setul de date ordonat crescător care îl împarte în jumătate. Astfel, 50% din valori vor fi mai mici decât mediana și 50% vor fi mai mari. Mediana oferă informații importante despre setul de date, deoarece, spre deosebire de medie, nu este afectată de valorile “aberrante”, adică valorile mult mai mici sau, dimpotrivă, mult mai mari decât majoritatea valorilor din setul de date. Pentru valori mici ale lui p , algoritmul pentru Varianta 1 a returnat valori ale medianei asemănătoare cu media, însă, odată cu creșterea lui p , mediana a rămas considerabil mai mică decât media. De exemplu, pentru $p = 1034317, g = 2, h = 735679$, mediana este 19337, în timp ce media de 26869,95 a fost afectată de valori extreme mari. În cazul Variantei 2, media și mediana și-au menținut valori apropiate pentru toate datele inițiale testate, ceea ce denotă o distribuție simetrică a datelor.

Deviația standard este un indicator al dispersiei valorilor. Deoarece generarea de numere aleatoare reprezintă un aspect important care stă la baza celor doi algoritmi, este de înțeles de ce amândouă variantele de implementare au afișat valori destul de mari ale deviației standard. Procesul aleator are drept consecință impredictibilitatea: uneori este necesar un număr mic de teste și algoritmul rulează rapid, alte ori este foarte lent din cauza numărului mare de teste efectuate, rezultând o performanță instabilă. Cu toate acestea, metoda 2 a demonstrat în continuare o stabilitate mai bună decât metoda 1, cu valori ale deviației standard mult mai mici.

Pentru o înțelegere mai aprofundată, am calculat, de asemenea, raportul dintre deviația standard și medie. Varianta 2 a obținut valori foarte apropiate pentru toate datele de intrare, cuprinse între 0,51 și 0,56, în timp ce Varianta 1 a avut cel mai bun rezultat aproximativ 0.93, ajungând chiar și la valoarea 1.01, pentru $p = 673, g = 5, h = 211$, caz în care deviația standard a depășit media. Pentru a avea un algorithm eficient și stabil, ne dorim ca acest raport să fie cât mai mic cu putință, așadar, și în acest caz, Varianta 2 este de preferat Variantei 1.

În ceea ce privește rezultatul minim al celor două variante, observăm că valorile sunt, în general, destul de apropiate. Sunt cazuri în care Varianta 1 a avut mai puține numere testate, dar și alte cazuri în care Varianta 2 a performat mai bine. Dacă am face o analiză

ce privește numai numărul minim de numere testate de cei doi algoritmi, nu am avea niște rezultate concludente, deoarece intervine factorul determinat de noroc. De aceea, este importantă o analiză complexă care să cuprindă mai mulți factori de comparație.

Pe de altă parte, pentru toate datele inițiale testate, Varianta 2 a returnat numărul maxim de numere testate mai mic decât Varianta 1, fiind astfel mai eficientă. Faptul că minimul și maximul returnate de Varianta 2 sunt mai apropiate de valoarea mediei este explicat și de deviația standard, ce denotă o împrăștiere mai restrânsă a valorilor față de Varianta 1. Efectuăm o analiză mai aprofundată în următoarea secțiune.

4.3.2 Analiza boxplot

Pentru a vizualiza mai bine datele, am analizat, în plus, mediana, prima și a treia quartilă și am reprezentat datele grafic sub forma unui boxplot.

În statistică, reprezentarea datelor prin intermediul unei diagrame **boxplot** este o modalitate de a observa cum sunt localizate datele, cât sunt de dispersate valorile și, posibil, asimetria distribuției datelor [16]. Toate aceste informații sunt redade cu ajutorul următoarelor elemente: minimul, maximul, mediana, prima quartilă și a treia quartilă.

Componentele diagramei sunt:

- **cutia**, ce cuprinde 50% din valori: este delimitată de prima și a treia quartilă și, în plus, are trasată o linie orizontală ce reprezintă valoarea mediane. În funcție de cum sunt distribuite datele, mediana poate să nu se afle la mijlocul cutiei;
- **mustățile**, ce surprind valorile din exteriorul cutiei, până la o anumită limită. Marginea superioară a mustăților este cea mai mare valoare în limita a $1,5 \times$ distanța dintre prima și a treia quartilă. Analog, marginea inferioară este cea mai mică valoare în limita a $1,5 \times$ distanța dintre prima și a treia quartilă. De aceea, mustățile pot să nu fie simetrice. Există cazuri în care aceste limite sunt chiar valorile minime și maxime din setul de date, atunci când observațiile sunt mai restrânse;
- **valorile aberante**, acele observații care se află în afara limitei mustăților.

Cu cât cutia este mai îngustă (adică diferența dintre prima și a treia quartilă este mai mică), cu atât algoritmul este mai stabil. De asemenea, multe valori aberante denotă caracterul imprevizibil al algoritmului, datorat generării de numere aleatoare.

Studiem, de exemplu, cazul $p = 611953, g = 7, h = 78902$.

Prima variantă implementată a obținut următoarele valori: *media* 21854.73, *mediana* 16146, *minimul* 548, *maximul* 130745, *prima quartilă* 7053, *a treia quartilă* 29466.

A doua variantă implementată a obținut *media* 4171.77, *mediana* 3789, *minimul* 594, *maximul* 18093, *prima quartilă* 2534, *a treia quartilă* 5390.

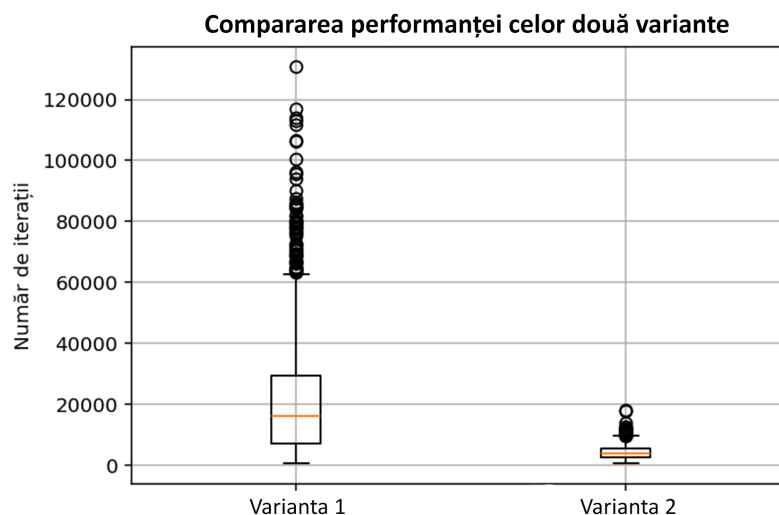


Figura 4.1: Diagrama boxplot pentru cele două variante implementate

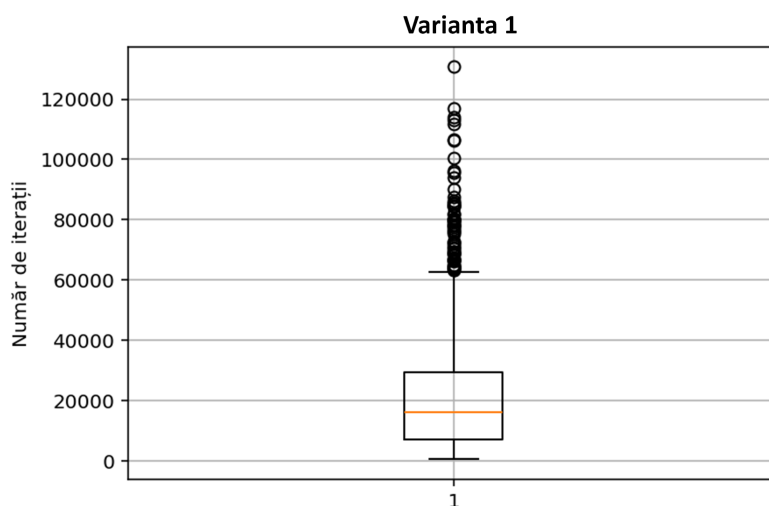


Figura 4.2: Diagrama boxplot pentru Varianta 1

Putem observa în Figura 4.1 că mediana Variantei 2 este situată mai jos decât mediana Variantei 1, ceea ce, încă de la început, este un semn că varianta a doua este mai rapidă. De asemenea, dimensiunea cutiei corespunzătoare Variantei 1 este foarte mare, ceea ce înseamnă că dispersia datelor centrale (cuprinse între prima și a treia cuartilă) este ridicată. În alte cuvinte, performanța variază mult chiar și între cazurile tipice. Comparativ, cutia corespunzătoare Variantei 1 este mult mai mică, ceea ce indică o consistență mai mare în numărul de teste efectuate în prima etapă a metodei indexului.

Observăm în cazul primei variante de implementare că există foarte multe valori aberante, maximul depășind chiar valoarea de 130.000. A doua variantă are și ea valori aberante, însă mai puține ca număr și mai apropiate de limita superioară a mustăților (sub 20.000). Așa cum am menționat anterior în această lucrare, prezența valorilor

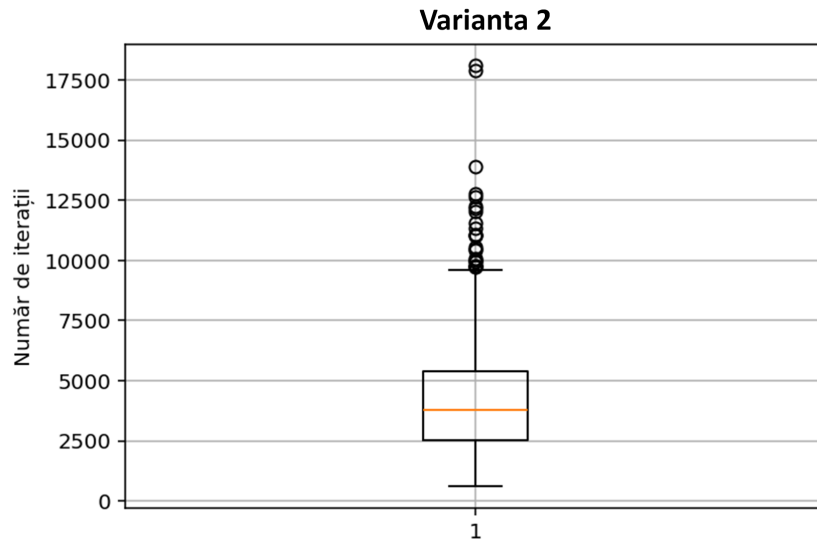


Figura 4.3: Diagrama boxplot pentru Varianta 2

aberrante este puternic corelată cu generarea de numere aleatoare.

În cazul ambelor variante implementate, distribuția valorilor este asimetrică, cu o răspândire semnificativ mai mare către valorile mari. Acest lucru înseamnă că, deși majoritatea cazurilor au valori rezonabile, în unele execuții algoritmul poate deveni inefficient. De asemenea, modul în care sunt răspândite observațiile explică valorile crescute ale deviației standard discutate anterior.

Așadar, Varianta 2 este mult mai eficientă și stabilă, în timp ce Varianta 1, deși uneori performantă, poate necesita un număr foarte mare de iterații, ceea ce o face mai riscantă în practică.

Pentru toate datele inițiale testate, rezultatele au fost asemănătoare cu acest caz particular.

Capitolul 5

DLP în alte grupuri

În acest capitol studiem comportamentul problemei logaritmului discret în alte grupuri, precum $\mathbb{F}_{2^k}^*$, și în contextul curbelor eliptice. Referința principală consultată în acest capitol este [15].

5.1 Metoda indexului în $\mathbb{F}_{2^k}^*$

Conform Teoremei 1.1.11, există un corp finit cu p^n elemente, pentru orice p număr prim și $n \geq 1$ natural. Mai mult, oricare două corpuri finite cu același număr de elemente sunt izomorfe. Procedul de construcție a unui astfel de corp derivă din următoarea teoremă:

Teorema 5.1.1. *Fie K un corp și fie $f(X) \in K[X]$ un polinom ireductibil de grad n . Atunci inelul factor $\frac{K[X]}{(f(X))} = L$ este un corp, iar $K \subseteq L$ este o extindere de grad n .*

Așadar, pentru a găsi un corp cu p^n elemente, este suficient să găsim un polinom $f(X)$ ireductibil de grad n în $\mathbb{Z}_p[X]$, iar $\frac{\mathbb{Z}_p[X]}{(f(X))} \simeq \mathbb{F}_{p^n}$ va fi corpul căutat. În plus, $\mathbb{F}_{p^n}$ este o extindere de grad n a lui $\mathbb{Z}_p$, deci $\mathbb{Z}_p$ este un subcorp al său.

Caracteristica unui corp $\mathbb{F}_q$ este cel mai mic întreg pozitiv s astfel încât suma a s copii ale lui $1 \in \mathbb{F}_q$ este egală cu $0 \in \mathbb{F}_q$. Orice corp finit $\mathbb{F}_q$ are $q = p^n$, iar $\mathbb{F}_{p^n}$ este o extindere a lui $\mathbb{Z}_p$, prin urmare, caracteristica lui $\mathbb{F}_{p^n}$ este p .

Cu această construcție, știind că $\mathbb{F}_{p^n}^*$ este un grup ciclic împreună cu operația de înmulțire, avem oportunitatea de a extinde metoda indexului în grupuri de această formă. Algoritmul trebuie însă modificat în funcție de grupul în care se încearcă rezolvarea problemei logaritmului discret, iar p și n trebuie să îndeplinească condiții suplimentare. În alte cuvinte, nu există, la momentul actual, o variantă a metodei indexului care să fie fezabilă pentru orice grup $\mathbb{F}_{p^n}^*$.

În practică, cele mai studiate corpuri în vederea dezvoltării unui algoritm de tip metoda indexului sunt corpurile $\mathbb{F}_{2^n}$. Conform [15], Thomé a publicat în 2001 o adaptare a metodei indexului ce rezolvă DLP în $\mathbb{F}_{2^{607}}^*$. În 2013, Antoine Joux a introdus o nouă

variantă a metodei indexului, care îmbunătățește eficiența calculului logaritmilor discreți în $\mathbb{F}_{2^n}^*$, în special pentru valori compuse ale lui n . Această abordare a fost perfecționată ulterior, iar în 2014 Robert Granger, Thorsten Kleinjung și Jens Zumbrägel au anunțat că au folosit această metodă pentru a calcula cu succes logaritmi discreți în $\mathbb{F}_{2^{9234}}^*$.

Mai departe, vom prezenta succint algoritmul dezvoltat de Joux.

Metoda indexului a lui Joux

Rezolvarea problemei logaritmului discret în $\mathbb{F}_{2^n}^*$ se bazează pe observația că, dacă $2^n = (q^2)^k$, atunci $\mathbb{F}_{2^n}$ poate fi văzut ca o extindere de grad k a corpului $\mathbb{F}_{q^2}$. Pentru ca algoritmul lui Joux să fie eficient, n și k trebuie să fie aproximativ la fel de mari, altfel, adaptări suplimentare ale metodei sunt necesare. Urmărim prezentarea din [15, Chapter 7.4.1].

Elementele lui $\mathbb{F}_{2^n} = \mathbb{F}_{(q^2)^k}$ sunt privite ca polinoame peste $\mathbb{F}_{q^2}$, iar Joux ia ca bază de factorizare toate polinoamele din $\mathbb{F}_{q^2}[x]$ care au grad cel mult 2. Acest pas este similar alegerii bazei de numere prime din algoritmul pentru metoda indexului studiat în această lucrare.

Spre deosebire de metoda indexului în $\mathbb{F}_p^*$, unde căutam numere care să aibă în factorizarea lor exclusiv numerele prime din baza aleasă, Joux dezvoltă o modalitate de a construi polinoame care să aibă factorizarea dorită. Observăm că elementele din $\mathbb{F}_q^*$ formează un subgrup al lui $\mathbb{F}_{q^2}^*$ de ordin $q - 1$. Astfel, orice element $\alpha \in \mathbb{F}_q^*$ satisface $\alpha^{q-1} = 1$, și, prin urmare, $\alpha^q = \alpha$. Așadar, cele q elemente ale lui $\mathbb{F}_q$ sunt toate rădăcini ale polinomului $x^q - x \in \mathbb{F}_{q^2}[x]$. Deci, în $\mathbb{F}_{q^2}$, $\alpha^q - \alpha = 0$. Urmând această observație, putem scrie în $\mathbb{F}_{q^2}[x]$:

$$x^q - x = \prod_{\alpha \in \mathbb{F}_q} (x - \alpha). \quad (5.1)$$

Ecuția (5.1) prezintă $x^q - x$ în $\mathbb{F}_{q^2}[x]$ ca un produs de elementele din baza aleasă. Ne vom folosi de această relație pentru a găsi polinoame în $\mathbb{F}_{2^n}$ care se descompun în funcție de factorii din bază. Pentru a realiza acest lucru, trebuie să găsim un polinom P de grad k ireductibil în $\mathbb{F}_{q^2}[x]$, care să dividă $h_1(x)x^q - h_2(x)x$, unde $h_1(x), h_2(x) \in \mathbb{F}_{q^2}[x]$ sunt două polinoame alese în mod convenabil. Cu aceste alegeri, în corpul $\frac{\mathbb{F}_{q^2}[x]}{(P)} \simeq \mathbb{F}_{2^n}$ avem $x^q = h_2(x)x/h_1(x)$ și $x^q - x = x(\frac{h_2(x)}{h_1(x)} - 1)$. Dacă, pentru P , $h_1(x)$ și $h_2(x)$ alese în mod eficient, $\frac{h_2(x)}{h_1(x)}$ este un polinom cu un grad suficient de mic, atunci $x^q - x$ se descompune în funcție de elementele din baza aleasă anterior.

Mai departe, pentru a genera un nou astfel de element, Joux demonstrează că este suficientă aplicarea unei schimbări de variabile atent alese. Astfel, se repetă procedeul de câte ori este necesar pentru a aplica apoi metode de algebră liniară pentru a afla logaritmii factorilor din bază, în mod similar procedului din metoda indexului obișnuită. Spre deosebire de metoda prezentată în această lucrare, în a doua etapă a metodei indexului,

ce presupune aflarea logaritmului căutat, Joux prezintă o nouă abordare, pe care a numit-o *metodă descendantă*. Totuși, nu vom prezenta acest algoritm în această lucrare, însă întreaga metodă poate fi găsită în [6].

5.2 DLP pe curbe eliptice

Pentru a înțelege DLP în cazul curbelor eliptice, este necesar să introducem următoarele definiții și teoreme, preluate din [15]:

Definiția 5.2.1. Fie K un corp comutativ de caracteristică diferită de 2 sau 3. O curbă eliptică peste K este o mulțime de forma:

$$E(K) = \{(X, Y) \in K \times K : Y^2 = X^3 + AX + B\} \cup \{\mathcal{O}\},$$

unde $A, B \in K$ și $\Delta = 4A^3 + 27B^2 \neq 0$. Δ se numește discriminantul curbei eliptice, iar $\mathcal{O}$ este un simbol nou, numit “punctul de la infinit”. Ecuația $Y^2 = X^3 + AX + B$ se numește ecuația lui Weierstrass.

Observația 5.2.2. Dacă un punct (X, Y) se află pe $E(K)$, atunci și simetricul acestuia față de axa Ox , $(X, -Y)$, aparține curbei eliptice.

Definim *operația de adunare* a două puncte din $E(K)$ după cum urmează. Pentru a putea vizualiza acest procedeu, considerăm $K = \mathbb{R}$. Fie $P_1(X_1, Y_1), P_2(X_2, Y_2) \in E(K), P_1 \neq P_2$. Trasăm dreapta P_1P_2 și considerăm al treilea punct de intersecție al dreptei cu curba E , după care luăm simetricul acestuia față de axa Ox . Notăm acest punct cu $P_1 \oplus P_2$. În cazul în care vrem să adunăm un punct P cu el însuși, procedeul este similar: trasăm tangenta la curbă în loc de dreapta P_1P_2 și continuăm cum am descris anterior.

În cazul în care $X_1 = X_2$ și $Y_1 = -Y_2$, adică P_1 și P_2 sunt simetrice față de Ox , atunci $P_1 \oplus P_2 = \mathcal{O}$. Acest punct $\mathcal{O}$ nu se află în planul xOy , dar considerăm că se află pe toate dreptele verticale ce se intersectează cu curba $E(K)$, motiv pentru care îl numim punct de la infinit. Astfel, punctul $\mathcal{O}$ acționează ca element neutru la adunare, deci $P \oplus \mathcal{O} = \mathcal{O} \oplus P = P$, iar opusul punctului $P(X_1, Y_1)$ este $P'(X_1, -Y_1)$, deoarece $P \oplus P' = \mathcal{O}$.

Teorema 5.2.3. Fie curba eliptică $E(K)$ dată de ecuația lui Weierstrass $Y^2 = X^3 + AX + B$ și $P_1(X_1, Y_1), P_2(X_2, Y_2) \in E(K)$ astfel încât $P_1, P_2 \neq \mathcal{O}$ și nu sunt simetrice față de Ox . Definim λ astfel:

$$\lambda = \begin{cases} \frac{Y_2 - Y_1}{X_2 - X_1} & \text{dacă } P_1 \neq P_2 \\ \frac{3X_1^2 + A}{2Y_1} & \text{dacă } P_1 = P_2 \end{cases}$$

și luăm:

$$X_3 = \lambda^2 - X_1 - X_2 \quad \text{și} \quad Y_3 = \lambda(X_1 - X_3) - Y_1.$$

Atunci $P_1 \oplus P_2 = (X_3, Y_3)$.

Teorema 5.2.4. *Curba eliptică $E(K)$ este grup abelian împreună cu operația de adunare $\oplus$ definită mai sus.*

Problema logaritmului discret

Pentru a putea defini problema logaritmului discret pe curbe eliptice, trebuie să ne uităm la o formă particulară a acestora și anume $E(K)$, unde $K = \mathbb{F}_p$. În acest caz, $(E(K), \oplus)$ este grup abelian finit.

Definiția 5.2.5. Fie E o curbă eliptică peste corpul finit $\mathbb{F}_p$, unde p este un număr prim, $p > 3$, și fie P și Q diferite de $\mathcal{O}$, două puncte din $E(\mathbb{F}_p)$. *Problema logaritmului discret pe curbe eliptice* presupune găsirea unui întreg n astfel încât $Q = nP$, unde $nP = P \oplus \dots \oplus P$ de n ori. Prin analogie cu problema logaritmului discret pentru $\mathbb{F}_p^*$, notăm acest întreg n prin:

$$n = \log_P(Q)$$

și numim n *logarithmul discret eliptic* al lui Q față de P .

Observația 5.2.6. Pot exista puncte $P, Q \in E(\mathbb{F}_p)$ astfel încât Q să nu fie un multiplu al lui P . În acest caz, $\log_P(Q)$ nu este definit.

Observația 5.2.7. Dacă există o valoare n care satisface $Q = nP$, atunci există o infinitate de astfel de valori. Deoarece $E(\mathbb{F}_p)$ este finit, punctele din lista $P, 2P, 3P, 4P, \dots$ nu pot fi toate distincte. Așadar, există întregi $k > j$ astfel încât $kP = jP$, și putem lua $s = k - j$. Cel mai mic astfel de $s \geq 1$ se numește *ordinul* lui P . În plus, ordinul lui P divide $|E(\mathbb{F}_p)|$. Astfel, dacă s este ordinul lui P și n_0 este un întreg astfel încât $Q = n_0P$, atunci toate soluțiile pentru $Q = nP$ sunt întregii de forma $n = n_0 + ks$, cu $k \in \mathbb{Z}$ și alegem $\log_P(Q)$ egal cu n_0 , unica soluție din intervalul $[0, s - 1]$.

Operația de adunare a punctelor de pe o curbă eliptică este costisitoare. Există însă un algoritm care scurtează acest procedeu. Acesta se numește „Dublează și Adună” și este similar cu Algoritmul de exponențiere rapidă (1) prezentat în Capitolul 4.1. Și de această dată se scrie descompunerea binară a numărului $n = a_0 + a_1 \cdot 2 + a_2 \cdot 2^2 + \dots + a_r \cdot 2^r$, cu $a_i \in \{0, 1\}, i = 1, \dots, r$, însă, de această dată, adunăm punctul P cu el însuși, în loc de a-l ridica la putere.

Pentru a rezolva problema logaritmului discret în $E(\mathbb{F}_p)$, există mai mulți algoritmi. Unul dintre ei este adaptarea algoritmului lui Shanks, *Baby Steps - Giant Steps* la curba eliptică $E(\mathbb{F}_p)$. Conform [5, Chapter 6.3.2], pentru a rezolva $Q = nP$, alegem în mod aleator întregii $j_1, \dots, j_r$ și $k_1, \dots, k_r$ între 1 și p , și construim două liste de puncte:

Lista 1: $j_1P, j_2P, j_3P, \dots, j_rP$

Lista 2: $k_1P + Q, k_2P + Q, k_3P + Q, \dots, k_rP + Q$

Când găsim un element comun între cele două liste, algoritmul se oprește, deoarece, dacă $j_uP = k_vP + Q$, atunci $Q = (j_u - k_v)P$, ceea ce oferă soluția.

Conform [15], motivul principal pentru care curbele eliptice sunt utilizate în criptografie este datorat faptului că nu se cunosc algoritmi precum metoda indexului pentru rezolvarea problemei logaritmului discret pe curbe eliptice, și, în general, nu există algoritmi cunoscuți care să rezolve această problemă în mai puțin de $O(\sqrt{p})$ pași. Pentru p un număr prim și k un întreg, dacă există un izomorfism între subgrupul generat de un punct $P \in E(\mathbb{F}_p)$ și $\mathbb{F}_{p^k}^*$, sub anumite condiții, atunci problema logaritmului discret se poate muta din grupul curbelor eliptice în $\mathbb{F}_p^*$, unde este posibilă aflarea soluției prin algoritmi similari metodei indexului discutate în această lucrare. Menezes, Okamoto și Vanstone au arătat în lucrarea [9] că este posibil un astfel de algoritm pentru o anumită clasă de curbe eliptice, ce a reușit o rezolvare în timp subexponențial a problemei logaritmului discret în acest caz particular.

Bibliografie

- [1] Leonard M. Adleman, „A subexponential algorithm for the discrete logarithm problem with applications to cryptography”, în *Proceedings of the 20th Annual IEEE Symposium on Foundations of Computer Science (FOCS)* (1979), pp. 55–60.
- [2] T. M. Apostol, *Introduction to Analytic Number Theory*, Undergraduate Texts in Mathematics, Springer, 1976.
- [3] E. R. Canfield, P. Erdős și C. Pomerance, „On a problem of Oppenheim concerning ‘factorisatio numerorum’”, în *Journal of Number Theory* 17.1 (1983), pp. 1–28.
- [4] Keith Conrad, *A Multivariable Hensel’s Lemma*, <https://kconrad.math.uconn.edu/blurbs/gradnumthy/multivarhensel.pdf>, Accesat la data: 2025-05-27.
- [5] J. Hoffstein, J. Pipher și J.H. Silverman, *An Introduction to Mathematical Cryptography*, Undergraduate Texts in Mathematics, Springer New York, 2014.
- [6] Antoine Joux, „A New Index Calculus Algorithm with Complexity $L(1/4+o(1))$ in Small Characteristic”, în *Advances in Cryptology – EUROCRYPT 2013*, vol. 7881, Lecture Notes in Computer Science, Springer, 2013, pp. 355–379.
- [7] Brian A. LaMacchia și Andrew M. Odlyzko, „Solving large sparse linear systems over finite fields”, în *Proceedings of CRYPTO 1991*, Springer, 1991, pp. 109–133.
- [8] Ueli M. Maurer și Andrew M. Odlyzko, „Discrete logarithms and the Diffie-Hellman problem”, în *Proceedings of CRYPTO 1985* (1985), pp. 448–457.
- [9] Alfred Menezes, Scott Vanstone și Tatsuaki Okamoto, „Reducing elliptic curve logarithms to logarithms in a finite field”, în *Proceedings of the twenty-third annual ACM symposium on Theory of computing*, 1991, pp. 80–89.
- [10] *Metoda indexului - implementare proprie în Python*, https://github.com/stefania1111/metoda_indexului, Disponibil online, Iunie 2025.
- [11] Laurențiu Panaitopol și Alexandru Gica, *O introducere în aritmetica și teoria numerelor*, Editura Universității din București, 2001.
- [12] Stephen C. Pohlig și Martin E. Hellman, „An Improved Algorithm for Computing Logarithms over $GF(p)$ and its Cryptographic Significance”, în *IEEE Transactions on Information Theory* 24.1 (1978), pp. 106–110.

- [13] J. M. Pollard, „Monte Carlo Methods for Index Computation (mod p)”, în *Mathematics of Computation* 32.143 (1978), pp. 918–924.
- [14] Daniel Shanks, „Class number, a theory of factorization, and genera”, în *Proceedings of Symposia in Pure Mathematics*, vol. 20, American Mathematical Society, 1971.
- [15] Douglas R. Stinson și Maura B. Paterson, *Cryptography: Theory and Practice*, a 4-a ed., CRC Press, 2019.
- [16] Wikipedia contributors, *Box plot* — *Wikipedia, The Free Encyclopedia*, Accesat la data: 2025-05-06, 2024, URL: https://en.wikipedia.org/wiki/Box_plot.

Anexa A

Codul pentru Varianta 1

Anexa prezintă codul Python utilizat pentru prima variantă de implementare a metodei indexului descrisă în Capitolul 4.2.1, corespunzătoare primei etape a algoritmului. Funcția prin care este implementată această etapă, *etapa1*($p, g, h, \mathcal{B}$), prezentată în A.2, se folosește de funcția *rezolvare_sistem* ($A, b, Ab, mod, putere$), expusă în A.1. Aceasta din urmă rezolvă sistemul de congruențe pătratic, compatibil determinat, modulo un factor de forma q^k , unde q este un număr prim, iar $k \in \mathbb{N}^*$, folosindu-se de Metoda Gauss, de Lema lui Hensel 1.3.2, dacă este cazul, și de Lema Chineză a Resturilor 1.2.3.

Codul complet pentru această variantă de implementare poate fi accesat prin [10].

```
1 def rezolvare_sistem(A, b, Ab, mod, putere):
2     if putere == 1:
3         x = met_gauss_modulo(A, Ab, mod)
4         return x
5     else:
6         x = met_gauss_modulo(A, Ab, mod)
7         x = np.array([x]).T
8         p = mod
9         A_inv = invers_matrice_modulo(A, p)
10        for i in range(2, putere+1):
11            omega = b - A@x
12            x = x + (A_inv @ omega)
13            x = x % (p**i)
14        return x.flatten()
```

A.1: Funcția care rezolvă sistemul de congruențe modulo p^k

```
1 def etapa1(p, g, h, B):
2     A, b, Ab = cautare_C_congruente(B, g, p)
3     ok = 0
4     while ok == 0:
5         if verificare_sistem(A, p-1):
```

```

6         ok = 1
7     else:
8         A, b, Ab = cautare_C_congruente(B, g, p)
9     fact = Factorizare(p-1)
10    lista_solutii = []
11    for prim in fact.keys():
12        mod = prim
13        putere = fact[prim]
14        sol = rezolvare_sistem(A, b, Ab, mod, putere)
15        lista_solutii.append((sol, mod**putere))
16    nr_necunoscute = len(B)
17    solutie_sistem = LCR(lista_solutii, nr_necunoscute, p-1)
18    logaritmi_baza = solutie_sistem
19    return logaritmi_baza

```

A.2: Funcția pentru prima etapă. Varianta 1

Anexa B

Codul pentru Varianta 2

Codul prezentat în această anexă ilustrează a doua variantă de implementare a metodei indexului, utilizată pentru rezolvarea primei etape a algoritmului, conform specificațiilor din Capitolul 4.2.2. Codul complet este disponibil online, accesând [10].

```
1 def etapa1(p, g, h, B):
2     nr_gasite = 0
3     nr_b_smooth = []
4     lista_exponenti = []
5     lista_ti = []
6     while nr_gasite < len(B):
7         ti = random.randint(0, p-2)
8         gi = exponentiere(g, ti, p)
9         valoare_adevar, factorizare = B_smooth(B, gi)
10        if valoare_adevar:
11            nr_b_smooth.append((gi, factorizare))
12            nr_gasite += 1
13            exponenti = np.zeros(len(B)).astype(int)
14            for i in factorizare.keys():
15                for prim in B:
16                    if i == prim:
17                        exponenti[B.index(prim)] = factorizare[i]
18            lista_exponenti.append(exponenti)
19            lista_ti.append(ti)
20        A = np.zeros((nr_gasite, len(B))).astype(int)
21        b = np.zeros((nr_gasite, 1)).astype(int)
22        for i in range(nr_gasite):
23            b[i, 0] = lista_ti[i]
24            for j in range(len(B)):
25                A[i, j] = lista_exponenti[i][j]
26        Ab = np.hstack((A, b))
```

```

27 fact = Factorizare(p-1)
28 lista_solutii = []
29 for prim in fact.keys():
30     mod = prim
31     putere = fact[prim]
32     copie_Ab = np.copy(Ab)
33     copie_Ab = copie_Ab % mod
34     coloana = 0
35     n = len(B)
36     while coloana < n:
37         linie_pivot = -1
38         for linie in range(coloana, n):
39             if copie_Ab[linie, coloana] % mod != 0:
40                 linie_pivot = linie
41                 pivot = copie_Ab[linie, coloana]
42                 break
43         if linie_pivot < 0:
44             am_gasit_pivot = False
45             while am_gasit_pivot == False:
46                 copie_Ab, Ab = adaugare_congruenta(copie_Ab, Ab,
47                 g, p, B, mod)
48                 if copie_Ab == 0:
49                     if copie_Ab[-1, 0] % mod != 0:
50                         pivot = copie_Ab[-1, 0] % mod
51                         am_gasit_pivot = True
52                         linie_pivot = len(copie_Ab) - 1
53                     else:
54                         for i in range(coloana):
55                             factor = copie_Ab[-1, i]
56                             copie_Ab[-1] = (copie_Ab[-1] - factor *
57                             copie_Ab[i]) % mod
58                             if copie_Ab[-1, coloana] % mod != 0:
59                                 pivot = copie_Ab[-1, coloana] % mod
60                                 am_gasit_pivot = True
61                                 linie_pivot = len(copie_Ab) - 1
62         if linie_pivot != coloana:
63             copie_Ab[[coloana, linie_pivot]] = copie_Ab[[
64                 linie_pivot, coloana]]
65             Ab[[coloana, linie_pivot]] = Ab[[linie_pivot, coloana
66             ]]
67             invers_pivot = invers_modulo(pivot, mod)

```

```

64     copie_Ab[coloana] = (copie_Ab[coloana] * invers_pivot) %
        mod # copie_Ab[coloana] este linia pivotului
65     for linie in range(len(copie_Ab)):
66         if linie != coloana:
67             factor = copie_Ab[linie, coloana]
68             copie_Ab[linie] = (copie_Ab[linie] - factor *
                copie_Ab[coloana]) % mod
69         coloana += 1
70     sol = copie_Ab[:n, -1]
71     if putere > 1:
72         sol = ridicare_solutie(sol, Ab, len(B), mod, putere)
73     lista_solutii.append((sol, mod**putere))
74 nr_necunoscute = len(B)
75 solutie_sistem = LCR(lista_solutii, nr_necunoscute, p-1)
76 logaritmi_baza = solutie_sistem
77 return logaritmi_baza

```

B.1: Funcția pentru prima etapă. Varianta 2

(Acest document se leagă împreună cu lucrarea de licență)

Declarație

Subsemnatul/Subsemnata GEORGESCU ȘTEFANIA, candidat la examenul de licență, sesiunea IUNIE-IULIE 2025, la Facultatea de Matematică și Informatică, programul de studii Matematici aplicate, declar pe propria răspundere că lucrarea de față este rezultatul muncii mele, pe baza cercetărilor mele și pe baza informațiilor obținute din surse care au fost citate și indicate, conform normelor etice, în note și în bibliografie. Declar că nu am folosit în mod tacit sau ilegal munca altora și că nici o parte din teză nu încalcă drepturile de proprietate intelectuală ale altcuiva, persoană fizică sau juridică. Declar că lucrarea nu a mai fost prezentată sub această formă vreunei instituții de învățământ superior în vederea obținerii unui grad sau titlu științific ori didactic.

Data

12.06.2025

Semnătura

